

Appunti del corso di Basi di dati

UniVR - Dipartimento di Informatica
secondo semestre - A.A. 2025/26

Mattia Nicolis Matteo Drago

Indice

1	Tecnologie per le basi di dati	1
1.1	Architettura transazionale di un DBMS	1
1.2	Transazione	2
1.2.1	Sintassi	2
1.2.2	Proprietà delle transazioni	3
1.3	Architettura di un DBMS	4
2	Linguaggio SQL	6
2.1	Istruzione SELECT	7
2.2	Clausola FROM	8
2.3	Clausola WHERE	9
2.3.1	Operatore LIKE	11
2.3.2	Operatore SIMILAR TO	11
2.3.3	Operatore BETWEEN	12
2.3.4	Operatore IN	12
2.3.5	Operatore IS NULL	13
2.4	SQL e ALGEBRA RELAZIONALE	14
2.4.1	Selezione e proiezione	14
2.4.2	SELECT: Senza proiezione	15
2.4.3	Proiezione: gestione dei duplicati in SQL	15
2.4.4	SELECT: Selezione, Proiezione, Join	16
2.4.5	Interrogazioni con JOIN	16
2.4.5.1	Interrogazioni con INNER JOIN	17
2.4.5.2	Interrogazioni con LEFT OUTER JOIN	17
2.4.5.3	Interrogazioni con RIGHT OUTER JOIN	18
2.4.5.4	Interrogazioni con FULL OUTER JOIN	18
2.4.6	Uso di variabili(alias)	20
2.4.7	Ordinamento del risultato	21
2.4.8	Operazioni Insiemistiche	22
2.4.8.1	Unione	22
2.4.8.2	Intersezione	23
2.4.8.3	Differenza	23
2.4.9	Operatori aggregati	24
2.4.9.1	COUNT	24
2.4.9.2	SUM, MAX, MIN, AVG	25
2.4.10	Interrogazione con Raggruppamento	27

2.4.11	Interrogazioni nidificate	29
2.4.11.1	Interrogazioni nidificate: WHERE	29
2.4.11.2	Interrogazioni nidificate: Binding e EXISTS	31
2.4.11.3	Interrogazioni nidificate: SELECT	32
2.4.11.4	Interrogazioni nidificate: FROM	33
3	Strutture fisiche e di accesso ai dati	34
3.1	Gestore del buffer	34
3.1.1	Buffer	35
3.1.2	Politica di gestione della memoria	35
3.1.3	Gestione delle pagine	35
3.1.4	Primitive per la gestione del buffer	36
3.1.4.1	Primitiva fix	36
3.1.4.2	Primitiva unfix	36
3.1.4.3	Primitiva setDirty	36
3.1.4.4	Primitiva force	36
3.1.4.5	Primitiva flush	37
3.2	Gestore dell'affidabilità	37
3.2.1	File di LOG	37
3.2.2	Regole di scrittura sul file di log	38
3.2.3	Guasti	39
3.3	Gestore dei metodi di accesso	41
3.3.1	Metodi di accesso	41
3.3.2	Organizzazione di una pagina	42
3.3.3	Operazioni sulle pagine	43
3.3.4	Strutture primarie per l'organizzazione dei file	43
3.3.4.1	Struttura sequenziale	43
3.3.4.1.1	Struttura sequenziale disordinata	43
3.3.4.1.2	Struttura sequenziale ad array	44
3.3.4.1.3	Struttura sequenziale ordinata	44
3.3.4.1.4	Indice	45
4	Laboratorio	48
4.1	Creazione di una tabella	48
4.1.1	Identificatori	49
4.1.2	Domini	49
4.1.2.1	Tipo carattere	49
4.1.2.2	Tipo boolean	50
4.1.2.3	Tipo numerici esatti	50
4.1.2.4	Tipo numerici approssimati	50
4.1.2.5	Istanti temporali	50
4.1.2.6	Intervalli temporali	51
4.1.2.7	Domini definiti dall'utente	51
4.1.3	Vincoli intrarelazionali	51
4.1.4	Vincoli interrelazionali	52
4.2	Modifica degli schemi	53
4.2.1	ALTER: modifiche alla struttura di una tabella	53
4.3	Cancellazione dati in una tabella	54
4.4	Inserimento e aggiornamento di tuple	54

4.5	Selezione	55
4.5.1	Clausola SELECT	55
4.5.1.1	Target List	56
4.5.1.2	Concatenazione stringhe	56
4.5.2	Clausola FROM	56
4.5.2.1	Uso di alias	57
4.5.2.2	Target List	57
4.5.3	Clausola WHERE	57
4.5.3.1	Operatore LIKE	58
4.5.3.2	Operatore SIMILAR TO	58
4.5.3.3	Operatore BETWEEN	58
4.5.3.4	Operatore IN	59
4.5.3.5	Operatore IS NULL	59
4.5.3.6	Operatore ORDER BY	59
4.5.3.7	Operatori di aggregazione	59
4.6	Esempi di interrogazioni	60
4.6.1	Interrogazione con raggruppamento: Group by	60
4.6.2	Interrogazione con JOIN	62
4.6.3	Interrogazioni con LEFT OUTER JOIN	62
4.6.4	Interrogazioni con RIGHT OUTER JOIN	62
4.6.5	Interrogazioni con FULL OUTER JOIN	62

Tecnologie per le basi di dati

Definizione: DBMS

Un **Database Management System** è un software in grado di gestire collezioni di dati che siano **grandi**, **condivise** e **persistenti**, assicurando la loro **affidabilità** e **privatezza**.

- ▷ **grandi**: dati devono essere organizzati in modo efficiente per poter essere gestiti in grandi quantità
- ▷ **condivise**: dati devono essere accessibili da più utenti contemporaneamente
- ▷ **persistenti**: dati devono essere memorizzati in modo permanente, anche dopo la chiusura del programma
- ▷ **affidabili**: dati devono essere protetti da errori e guasti del sistema
- ▷ **privati**: dati devono essere protetti da accessi non autorizzati

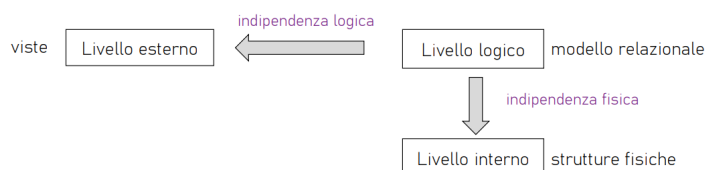
Un DBMS in quanto sistema software deve essere **efficiente** ed **efficace** e deve, inoltre, estendere le funzionalità del file system.

Definizione: base di dati

Una **base di dati** è una collezione di dati gestita da un DBMS.

1.1 Architettura transazionale di un DBMS

Un DBMS è costituito tra 3 livelli:



Un DBMS basato sul **modello relazionale** è nella maggior parte dei casi anche un sistema transazionale: fornisce un meccanismo per la definizione ed esecuzione di transazioni.

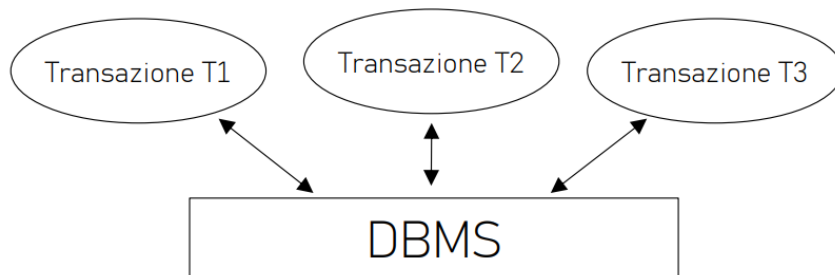


Figura 1.1: Ssistema transazionale

1.2 Transazione

Definizione: Transazione

Una **transazione** è un'unità di lavoro svolta da un programma applicativo (che interagisce con una base di dati) per la quale si vogliono garantire proprietà di correttezza, robustezza e isolamento.

Le caratteristiche principali sono:

- ▷ **atomicità**: una transazione deve essere eseguita completamente o non essere eseguita affatto
- ▷ **non sono ammesse esecuzioni parziali**

1.2.1 Sintassi

La sintassi per definire una transazione in SQL:

```
<transazione> -> begin transaction
                  <programma>
                  end transaction

<programma> -> < <istruzione> | commit work | rollback work >
               { < <istruzione> | commit work | rollback work > }
```

La transazione va a buon fine all'esecuzione di un `commit work`, ma non ha alcun effetto se viene eseguito un `rollback work`.

Per essere ben formata, una transazione deve:

- **iniziare** con un `begin transaction`
- **terminare** con un `end transaction`
- **esecuzione** comportare il raggiungimento di un `commit` o di un `rollback work`, a seguito del quale non si eseguono altri accessi alla base di dati

```
begin transaction;
  update CONTO set saldo = saldo - 1200
    where filiale = '005' and numero = 15;
  update CONTO set saldo = saldo + 1200
    where filiale = '005' and numero = 205;
  commit work;
end transaction;
```

Listing 1.1: esempio di transazione

1.2.2 Proprietà delle transazioni

Una transazione ha 4 proprietà fondamentali:

1. **atomicità**: una transazione è una unità di esecuzione indivisibile (viene eseguita completamente o non viene eseguita affatto).

Questo implica:

- ◇ se una transazione viene interrotta prima del `commit`, il lavoro fin qui eseguito dalla transazione deve essere disfatto ripristinando la situazione in cui si trovava la base di dati prima dell'inizio della transazione
- ◇ se una transazione viene interrotta dopo l'esecuzione del `commit` (`commit` eseguito con successo), il sistema deve assicurare che la transazione abbia effetto sulla base di dati

2. **consistenza**: esecuzione di una transazione non deve violare i vincoli di integrità.

Questo implica:

- ◇ verifica immediata:
 - fatta nel corso della transazione
 - viene abortita solo l'ultima operazione e il sistema restituisce all'applicazione una segnalazione d'errore
 - applicazione può reagire alla violazione
- ◇ verifica differita:
 - al `commit` se un vincolo di integrità viene violato la transazione viene abortita senza possibilità da parte dell'applicazione di reagire alla violazione

3. **isolamento**: esecuzione di una transazione deve essere indipendente dalla contemporanea esecuzione di altre transazioni

- `rollback` di una transazione non deve creare `rollback` a catena di altre transazioni che si trovano in esecuzione contemporaneamente
- sistema deve regolare l'esecuzione concorrente con meccanismi di controllo dell'accesso alle risorse

4. **persistenza**: effetto di una transazione che ha eseguito il `commit` non deve andare perso

- sistema deve essere in grado, in caso di guasto, di garantire gli effetti delle transazioni che al momento del guasto avevano già eseguito un `commit`

Un DBMS che gestisce transazioni dovrebbe garantire per ogni transazione che esegue tutte queste proprietà.

1.3 Architettura di un DBMS

L'architettura di un DBMS è composta da **moduli** che si occupano di gestire le diverse funzionalità del sistema.

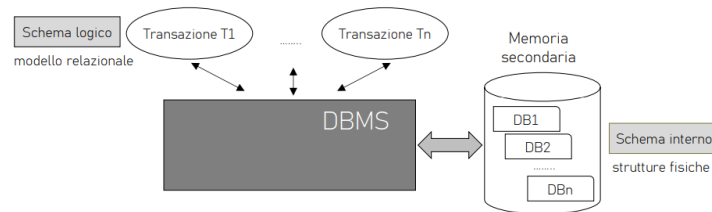


Figura 1.2: architettura di riferimento di un DBMS

Il modulo di gestione delle transazioni occupa di trasformare una transazione in un insieme di operazioni sulla base di dati, garantendo le proprietà ACID.

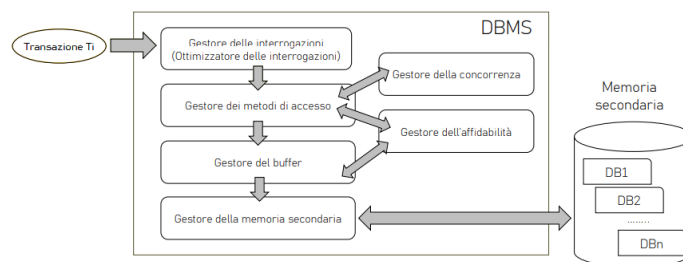
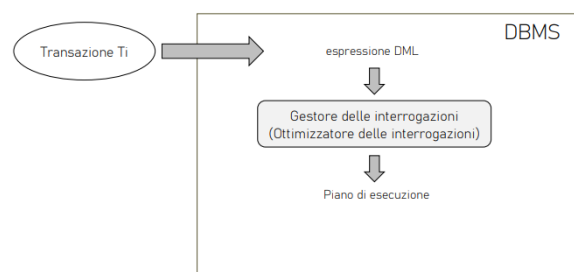


Figura 1.3: modulo di gestione delle transazioni di un DBMS

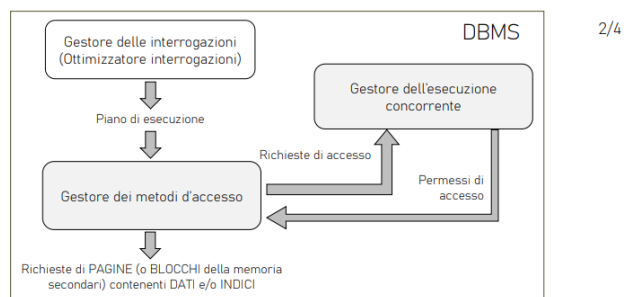
I singoli moduli che compongono questo modulo sono:

- ▷ **gestore delle interrogazioni:** riceve in ingresso una query SQL

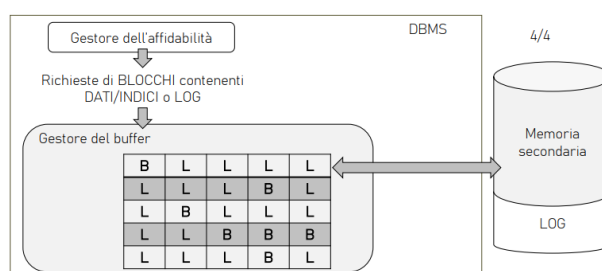


1/4

- ▷ **gestore dei metodi di accesso:** determina quali sono le pagine della memoria secondaria di cui si ha bisogno, ma capisce anche se è possibile accederci



- ▷ **gestore dell'affidabilità**: carica in memoria principale le pagine della memoria secondaria di cui si ha bisogno, e le mantiene in memoria finché non sono più necessarie



- ▷ **gestore del buffer**
- ▷ **gestore dell'esecuzione concorrente**
- ▷ **gestore dell'affidabilità**

I moduli che contribuiscono a garantire le proprietà ACID sono:

- **gestore dei metodi d'accesso**: consistenza
- **gestore dell'esecuzione concorrente**: atomicità e isolamento
- **gestore dell'affidabilità**: atomicità e persistenza

Linguaggio SQL

Il termine SQL in origine era l'acronimo di **S**tructured **Q**uery **L**anguage, ma ad oggi è diventato un nome proprio.

Definizione: linguaggio SQL

Il **linguaggio di interrogazione SQL** permette di:

- ▷ **definire delle strutture dati e i vincoli di integrità**: **D**ata **D**efinition **L**anguage
- ▷ **manipolare i dati** (inserimento, aggiornamento e cancellazione): **D**ata **M**anipulation **L**anguage
- ▷ **interrogare**: **D**ata **Q**uery **L**anguage

Esistono diverse versioni:

Nome informale	Nome ufficiale	Caratteristiche
SQL base	SQL-86	Costrutti base
	SQL-89	Integrità referenziale
SQL-2	SQL-92	Modello relazionale Vari costrutti nuovi 3 livelli: entry, intermediate, full
SQL-3	SQL:1999	Modello relazionale a oggetti Organizzato in diverse parti Trigger, funzioni esterne, ...
	SQL:2003	Estensioni del modello a oggetti Eliminazione di costrutti non usati Nuove parti: SQL/JRT, SQL/XML, ...
	SQL:2006	Estensione della parte XML
	SQL:2008	Lievi aggiunte (per esempio, trigger instead of)
	SQL:2011	Ricco supporto per dati temporali
	SQL:2016	Gestione del formato JSON

La parte di SQL dedicata alle dichiarazioni fa parte dei DML.

Le dichiarazioni vengono espresse in modo dichiarativo: specifica l'obiettivo e non la procedura per ottenerlo (algebra relazionale).

Una parte del DBMS, detta *query optimizer*, traduce di fatto l'interrogazione SQL nel linguaggio procedurale interno del sistema, che è nascosto all'utente.

SQL permette, quindi, di descrivere le interrogazioni in modo astratto ed ad alto livello.

2.1 Istruzione SELECT

La sintassi dell'istruzione SELECT è:

```
SELECT [ DISTINCT ]
[ * | expression [[ AS ] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

dove:

- ▷ **expression**: è un'espressione che determina un attributo
- ▷ **from_item**: è un'espressione che determina una sorgente per gli attributi
- ▷ **condition**: è un'espressione booleana per selezionare i valori degli attributi
- ▷ **grouping_element**: è un'espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato.

La struttura dell'istruzione è la seguente:

```
SELECT attributo
FROM tabella
[WHERE condizione]
```

SELECT, FROM e WHERE sono chiamate **clausole**.

La clausola **SELECT** estrae dal prodotto cartesiano delle tabelle elencate nella clausola **FROM**, le righe che soddisfano la condizione specificata nella clausola **WHERE**.

Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola **FROM** ed, eventualmente, filtrata dalla clausola **WHERE** (se esiste), le cui colonne dipendono dalla lista degli attributi nella clausola **SELECT**.

Interrogazione con SELECT

Consideriamo la seguente base di dati:

Impiegati

Nome	Cognome	D part	Uffic o	St pend o	Città
Mario	Rossi	Amministrazione	10	48	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	48	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Galli	Direzione	7	73	Genova
Paolo	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Estrarre lo stipendio degli impiegati di cognome 'Rossi'.

```
SELECT Stipendio
FROM Impiegato
WHERE Cognome = 'Rossi';
```

Il risultato dell'interrogazione è la seguente tabella:

Il simbolo * permette di indicare tutti gli attributi delle tabelle.

```
SELECT *
FROM Impiegato;
```

L'attributo AS permette di rinominare un attributo (alias)

```
SELECT Stipendio AS Salario
FROM Impiegato;
```

2.2 Clausola FROM

La clausola FROM specifica le tabelle da cui estrarre i dati.

La sintassi dell'istruzione FROM è:

```
FROM from_item [, ...]
```

dove from_item può essere una delle seguenti clausole:

- ▷ **table_name** **[[AS] alias [(column_alias [, ...])]**: se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle.
Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati antepoendo il nome della tabella e un '.' (es. nomeTabella.nomeAttributo)
- ▷ **(other_select) [AS] nomeRisultato [(column_alias [...])]**: è una SELECT innestata.
Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
- ▷ **from_item [NATURAL] join_type from_item [ON condition]**: è la clausola JOIN.
Metodo più sofisticato di 1, che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle.
join_type specifica il tipo di join

Interrogazione con FROM

Consideriamo le seguenti tabelle:

Estrarre i nomi e il cognome degli impiegati e le città in cui lavorano

```
SELECT Impiegato.Nome, Impiegato.Cognome, Dipartimento.Città
FROM Impiegato, Dipartimento
WHERE Impiegato.Dipart = Dipartimento.Nome
```

2.3 Clausola WHERE

ERE condition

- La clausola **WHERE** ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori **AND**, **OR** e **NOT**
 - AND**: seleziona le righe in cui tutti i predicati sono veri
 - OR**: seleziona le righe in cui almeno un predicato è vero
 - NOT**: seleziona le righe in cui il predicato è falso
- Ciascun predicato semplice usa gli operatori **=**, **<>**, **<**, **>**, **<=** e **>=** per confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante, oppure un'altra espressione.
- Per esprimere la precedenza degli operatori **AND** e **OR** si fa uso della parentesi.

- Q3: Estrarre i nomi ed il cognome degli impiegati che lavorano nell'ufficio 20 del dipartimento Amministrazione.**

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Ufficio = 20 AND
      Dipart = 'Amministrazione'
```

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma



Nome	Cognome
Giovanni	Verdi

- Q4: Estrarre i nomi ed il cognome degli impiegati che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

```
SELECT Nome, Cognome
FROM Impiegato
WHERE Dipart = 'Amministrazione'
OR Dipart = 'Produzione'
```



Nome	Cognome
Mario	Rossi
Carlo	Bianchi
Giovanni	Verdi
Paola	Rosati
Marco	Franco

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

- Q5: Estrarre i nomi degli impiegati di cognome Rossi che lavorano dipartimento Amministrazione o nel dipartimento Produzione.

```
SELECT Nome
FROM Impiegato
WHERE Cognome = 'Rossi' AND
(Dipart = 'Amministrazione' OR
Dipart = 'Produzione')
```



Nome
Mario

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

2.3.1 Operatore LIKE

- Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore **LIKE** che consente verificare se una stringa rispetta un certo "pattern" (oppure **ILIKE** per case insensitive).
- Due caratteri speciali per il confronto **"_"** e **"%"**:
 - **"_"**: confronto con un carattere arbitrario
 - **"%"**: confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari
- **'_o%i'** = stringa che ha una o in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una i.

- **Q6: Estrarre gli impiegati che hanno un cognome che ha una "o" in seconda posizione e finisce con una "i".**

SELECT *
FROM Impiegato
WHERE Cognome LIKE '_o%i';



Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Rossi	Direzione	14	80	Milano
Paola	Rosati	Amministrazione	75	40	Venezia

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

2.3.2 Operatore SIMILAR TO

- L'operatore **SIMILAR TO** è un **LIKE** più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).
- Esempi di componenti di espressioni regolari:
 - **_** = 1 carattere qualsiasi.
 - **%** = 0 o più caratteri qualsiasi.
 - ***** = ripetizione del precedente match 0 o più volte.
 - **+** = ripetizione del precedente match UNA o più volte.
 - **n,m** = ripetizione del precedente match almeno n e non più di m volte.
 - **...** = ... è un elenco di caratteri ammissibili

Esempio

Studenti con cognome che inizia con 'A' o 'B', o 'D', o 'N' e finisce con 'a':

```
SELECT cognome, nome, città FROM Studente
WHERE cognome SIMILAR TO '[ABDN]{1}%a';
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

2.3.3 Operatore BETWEEN

Esempio

Tutti gli studenti che hanno matricola tra 'IN0002' e 'IN0004':

```
SELECT cognome, nome, matricola FROM Studente
WHERE matricola BETWEEN 'IN0002' AND 'IN0004';
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

2.3.4 Operatore IN

Nella clausola **WHERE** può apparire l'operatore **[NOT] IN** per testare l'appartenenza di un valore ad un insieme.

```
WHERE attributo [NOT] IN (valore, [, ...]);
```


Esempio

Tutti gli studenti che hanno matricola nell'elenco 'IN0001', 'IN0003' e 'IN0005'.

```
SELECT cognome, nome, matricola FROM Studente
WHERE matricola IN('IN0001 ', 'IN0003 ', 'IN0005 ');
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

2.3.5 Operatore IS NULL

- Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato **IS NULL**.
- **Attributo IS NULL**: ritorna true se l'attributo ha valore nullo.
- **IS NOT NULL**: è la negazione, ritorna true se l'attributo ha valore diverso da nullo.

```
WHERE attributo IS [NOT] IN NULL;
```

SQL-2 implementa la logica a 3 valori:

- Un predicato semplice restituisce il valore NULL quando uno qualsiasi dei termini del predicato ha valore nullo.
- Il predicato IS NULL è una eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL.

Esempio

Tutti gli studenti che **NON** hanno una città.

```
SELECT cognome, nome, città FROM Studente
WHERE città IS NULL;
```

matricola	cognome	nome	indirizzo	città	media
IN0001	Rossi	Marco	via nobile	Verona	27.50
IN0002	Paoli	Paolo	via dritta	Padova	28.6666
IN0003	Bianchi	Dante	via storta	VERONA	18.345
IN0004	Rossi	Matteo	via nobile	Verona	24.567
IN0005	Verdi	Luca	via nuova	Va	28.6666
IN0006	Nocasa	Caio			

2.4 SQL e ALGEBRA RELAZIONALE

Cerchiamo ora di capire la relazione con le corrispondenti interrogazioni procedurali in algebra relazionale.

Scrivere questa espressione in SQL:

```
SELECT T1.attributo1l, ..., Th.attributohm
FROM Tabella1 as T1, ..., Tabellah as Th
WHERE Condizione
```

Corrisponde a fare questa operazione in algebra relazionale:

$$\Pi_{T_1.attributo_{1l}, \dots, T_h.attributo_{hm}} (\sigma_{Condizione} (T_1 \bowtie \dots \bowtie T_h))$$

Esempi più concreti

Prendiamo queste tabelle come riferimento iniziale:

Maternità		Paternità		Persone		
Madre	Figlio	Padre	Figlio	Nome	Età	Reddito
Luisa	Maria	Sergio	Franco	Andrea	27	21
Luisa	Luigi	Luigi	Olga	Aldo	25	15
Anna	Olga	Luigi	Filippo	Maria	55	42
Anna	Filippo	Franco	Andrea	Anna	50	35
Maria	Andrea	Franco	Aldo	Filippo	26	30
Maria	Aldo			Luigi	50	40
				Franco	60	20
				Olga	30	41
				Sergio	85	35
				Luisa	75	87

Figura 2.1: Tabelle di riferimento

2.4.1 Selezione e proiezione

Nome e reddito delle persone con meno di trenta anni

Algebra relazionale: $\pi_{Nome, Reddito}(\sigma_{Età < 30}(Persone))$

SQL: SELECT Nome, Reddito
FROM Persone
WHERE età < 30

SELECT: alias di tabella o di attributo

Con ALIAS di Tabella

```
SELECT p.nome, p.reddito
FROM persone as p
WHERE p.Età < 30
```

Con ALIAS di attributo

```
SELECT p.nome as nomePersona, p.reddito as
redditoPersona
FROM persone as p
WHERE p.Età < '30'
```

2.4.2 SELECT: Senza proiezione

Nome, reddito ed età delle persone con meno di trenta anni

Algebra relazionale: $\sigma_{Età < 30}(Persone)$

SQL: SELECT *
FROM Persone
WHERE età < 30

2.4.3 Proiezione: gestione dei duplicati in SQL

Una significativa differenza tra SQL e algebra relazionale è la gestione dei duplicati. L'operazione di eliminazione delle righe doppie (ad esempio al seguito di una selezione con where) è molto costosa, quindi di default SQL mantiene le righe duplicate.

Il costrutto **DISTINCT** dopo la parola chiave SELECT permette di eliminare i duplicati. Il costrutto **ALL** permette invece di mantenerli, ma questa è già la scelta di default.

Vediamone un esempio

Impiegati

Cognome	Nome	Filiale
Neri	Mario	Napoli
Neri	Giorgio	Napoli
Rossi	Claudio	Roma

Trovare cognome e filiale di tutti gli impiegati

Algebra relazionale: $\pi_{Cognome, Filiale}(Impiegati)$

Output: {(Neri, Napoli), (Rossi, Roma)}

SQL

```
SELECT Cognome, Filiale
FROM Impiegati
```

Impiegati

Cognome	Filiale
Neri	Napoli
Neri	Napoli
Rossi	Roma

SQL

```
SELECT DISTINCT Cognome, Filiale
FROM Impiegati
```

Impiegati

Cognome	Filiale
Neri	Napoli
Rossi	Roma

- Q7: Estrarre le città delle persone il cui cognome è Rossi

- ```
SELECT Città
FROM Persona
WHERE Cognome = 'Rossi'
```



| Città  |
|--------|
| Verona |
| Verona |
| Milano |

- ```
SELECT DISTINCT Città
FROM Persona
WHERE Cognome = 'Rossi'
```



Città
Verona
Milano

Persona

CodFiscale	Nome	Cognome	Città
RSSMRA55B21T234J	Mario	Rossi	Verona
BNCCLR69T30H745Z	Carlo	Bianchi	Roma
RSSGNN41A31B344C	Giovanni	Rossi	Verona
RSSPRT75C12F205V	Pietro	Rossi	Milano

2.4.4 SELECT: Selezione, Proiezione, Join

L'istruzione SELECT, con una sola relazione nella clausola FROM permette di realizzare: **selezioni**, **proiezioni**, **ridenominazioni**.

Invece con più relazioni nella clausola FROM si realizzano join (e prodotti cartesiani)

SELECT	π Proiezione + ρ Ridenominazione (con AS)
FROM	$\bowtie$ Join(prodotto cartesiano)
WHERE	σ Selezione + $\bowtie$ Join(condizion)

2.4.5 Interrogazioni con JOIN

In SQL il join si può definire in due modi diversi

- Il primo non richiede l'uso di nuova sintassi, e specifica il legame tra le tabelle coinvolte a livello della clausola WHERE.
- Il secondo usa una sintassi riservata a livello di clausola FROM (useremo questo).

La clausola **FROM** ammette come argomento:

table_name [[AS] name]] [, ...]

Si è visto che se sono presenti due o più nomi di tabelle, si esegue il prodotto cartesiano tra tutte le tabelle e lo schema del risultato può contenere tutti gli attributi del prodotto cartesiano.

Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**.

A partire da SQL-2, esistono altri tipi di JOIN (join_type): **INNER JOIN**, **LEFT [OUTER] JOIN**, **RIGHT [OUTER] JOIN** e **FULL [OUTER] JOIN**.

Sintassi generale

table_name [NATURAL] join_type table_name [ON join_condition [, ...]].

dove **join_condition** è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola **WHERE**

Prodotto Cartesiano

- Formulazione con JOIN:

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I
      CROSS JOIN Reparto AS R;
```
- Formulazione con ';':

```
SELECT I. cognome , R. nomeRep
FROM Impiegato AS I, Reparto AS R;
```

cognome	nome
Rossi	Acquisti
Verdi	Acquisti
Rossi	Vendite
Verdi	Vendite

2.4.5.1 Interrogazioni con INNER JOIN

Sintassi generale

table1 [INNER] JOIN table2 ON join_condition [, ...]

Rappresenta il tradizionale θ join dell'algebra relazionale. Combina ciascuna riga r_1 di table1 con ciascuna riga di table2 che soddisfa la condizione della clausola **ON**

SQL

```
SELECT I.cognome , R.nomeRep,
      R.sede
FROM Impiegato AS I
      INNER JOIN Reparto AS R
      ON I.nomerep = R.nomerep;
```

cognome	nomerep	sede
Rossi	Acquisti	Verona
Verdi	Vendite	Verona

2.4.5.2 Interrogazioni con LEFT OUTER JOIN

Sintassi generale

```
table1 LEFT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un **INNER JOIN**. Poi, per ciascuna riga $r1$ di $table1$ che non soddisfa la condizione con qualsiasi riga di $table2$, si aggiunge una riga al risultato con i valori di $r1$ e assegnando **NULL** agli altri attributi..

SQL

```
INSERT INTO Reparto (nomerep, sede, telefono)
VALUES ('Finanza', 'Padova', '02 8028888');
SELECT R.nomeRep, I.cognome
FROM Reparto AS R
LEFT OUTER JOIN Impiegato AS I
ON I.nomerep = R.nomerep;
```

nomerep	cognome
Acquisti	Rossi
Vendite	Verdi
Finanza	

Nota

Il **LEFT [OUTER] JOIN** non è simmetrico!

Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

SQL

```
SELECT I.cognome, R.nomeRep
FROM Impiegato I LEFT OUTER JOIN
RepartoR
ON I.nomerep = R.nomerep
```

cognome	nomerep
Rossi	Acquisti
Verdi	Vendite

2.4.5.3 Interrogazioni con RIGHT OUTER JOIN

Sintassi generale

```
table1 RIGHT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

SQL

```
SELECT I.cognome , R.nomeRep
FROM Impiegato I RIGHT OUTER JOIN
RepartoR
ON I. nomerep = R. nomerep
```

nomerep	cognome
Acquisti	Rossi
Vendite	Verdi
Finanza	

2.4.5.4 Interrogazioni con FULL OUTER JOIN

Sintassi generale

```
table1 FULL [ OUTER ] JOIN table2 ON join_condition [, ...].
```

È equivalente a:

- INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.
- Non è equivalente a CROSS JOIN!

Esempio

Nome, Cognome di tutti gli impiegati e città del Dipartimento in cui lavora

- **SELECT i.Nome, i.Cognome, d.Città**
FROM impiegato as i, Dipartimento
WHERE i.Dipart = d.Nome

Prodotto cartesiano

Nota: le tabelle Impiegato e Dipartimento, hanno entrambe un attributo "nome", quindi devo specificare a quale mi riferisco

impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

Il join si può spostare nella clausola from, a livello delle tabelle (nello svolgere gli esercizi, useremo questa sintassi dedicata).

```
SQL:  SELECT ListaAttributi
      FROM Tabella1 JOIN Tabella2 ON ( CondizioneDiJoin )
      [WHERE AltraCondizione ]
```

```
SQL:  SELECT i.Nome, d.Città
      FROM Impiegato AS i JOIN Dipartimento as d ON ( i.Dipart =
      d.Nome )
```

- Q8: Selezionare i padri di persone che guadagnano più di 20

- $\Pi_{\text{Padre}}(\text{Paternità} \bowtie_{\text{Figlio=Nome}} \sigma_{\text{Reddito}>'20'}(\text{Persone}))$

- SELECT DISTINCT Padre
FROM Persone JOIN Paternità ON (Figlio = Nome)
WHERE Reddito > '20'

2.4.6 Uso di variabili(alias)

SQL permette di associare un nome alternativo (alias) alle tabelle che compaiono nella clausola FROM. Il nome viene usato per far riferimento alla tabella nel contesto dell'interrogazione

Motivi:

- Far riferimento alla tabella in modo compatto.
- Fare accesso più volte alla stessa tabella
- Q9: Estrarre tutti gli impiegati che hanno lo stesso cognome ma nome diverso, del dipartimento Produzione.

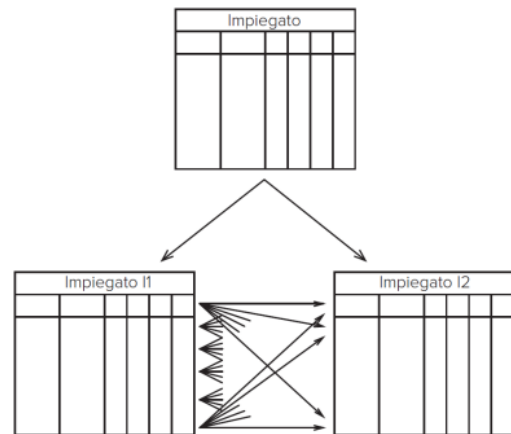
- SELECT i1.Cognome, I1.Nome
FROM Impiegato i1, Impiegato i2
WHERE i1.Cognome = i2.Cognome AND
i1.Nome <> i2.Cognome AND
i2.Dipart = 'Produzione'

Impiegato					
Nome	Cognome	Dipart	Ufficio	Stipendio	Città
Mario	Rossi	Amministrazione	10	45	Milano
Carlo	Bianchi	Produzione	20	36	Torino
Giovanni	Verdi	Amministrazione	20	40	Roma
Franco	Neri	Distribuzione	16	45	Napoli
Carlo	Rossi	Direzione	14	80	Milano
Lorenzo	Gialli	Direzione	7	73	Genova
Paola	Rosati	Amministrazione	75	40	Venezia
Marco	Franco	Produzione	20	46	Roma

Figura 2.2: Questa interrogazione confronta ciascuna riga di Impiegato con tutte le righe di Impiegato associate al dipartimento Produzione.

Funzionamento Si può immaginare che al momento della definizione dell'alias vengano create due copie della tabella Impiegato, ciascuna associata ad una delle due variabili i1 e i2.

- Le due copie contengono tutte le righe di Impiegato.
- Sulla copia i2 vengono selezionate solo le righe del dipartimento 'Produzione'.



2.4.7 Ordinamento del risultato

Come abbiamo visto, una relazione è un insieme di tuple non ordinato. Nella realtà, però, sorge spesso il bisogno di costruire un ordine sulle righe. SQL permette di specificare un ordinamento (ascendente o discendente) sulle righe dell'output grazie alla clausola ORDER BY che chiude l'interrogazione.

Sintassi generale

ORDER BY attributo [{ ASC | DESC }] [, ...];

Le righe vengono ordinate per il primo attributo nell'elenco, a parità di valore del primo attributo si usa il secondo, e così via in sequenza.

- Q10: Estrarre il nome, l'età ed il reddito delle persone con meno di trenta anni in ordine alfabetico

- SELECT Nome, Età, Reddito
FROM Persone
WHERE Età < '30'
ORDER BY Età

Nome	Età	Reddito		Nome	Età	Reddito
Andrea	27	21	➔	Aldo	25	15
Aldo	25	15		Filippo	26	30
Filippo	26	30		Andrea	27	21

- Q11: Estrarre il contenuto della tabella Automobile ordinato in base alla marca (in modo discendente) e al modello.

Automobile

Targa	Marca	Modello	NroPatente
KB 574 WW	Fiat	Punto	VR 2030020Y
GA 652 FF	Fiat	Panda	VR 2030020Y
BJ 747 XX	Lancia	Ypsilon	PZ 1012436B
ZB 421 JJ	Fiat	Uno	MI 2020030U

- **SELECT ***
FROM Automobile
ORDER BY Marca DESC, Modello



Targa	Marca	Modello	NroPatente
BJ 747 XX	Lancia	Ypsilon	PZ 1012436B
GA 652 FF	Fiat	Panda	VR 2030020Y
KB 574 WW	Fiat	Punto	VR 2030020Y
ZB 421 JJ	Fiat	Uno	MI 2020030U

2.4.8 Operazioni Insiemistiche

SQL mette a disposizione degli operatori insiemistici

SELECT UNION | INTERSECT | EXCEPT
[ALL] SELECT

Gli operatori insiemistici, al contrario del resto del linguaggio, assumono come default di eseguire un'eliminazione dei duplicati, a meno che si usi **ALL**.

Sono sufficienti domini compatibili ed ugual numero di attributi.

- **Ordine posizionale**: la corrispondenza degli attributi si basa sulla posizione
- Se i nomi degli attributi sono diversi di solito il sistema usa il nome del primo operando

2.4.8.1 Unione

- Q12: Estrarre i nomi ed i cognomi degli impiegati.

- **SELECT Nome**
FROM Impiegato
UNION
SELECT Cognome
FROM Impiegato



Nome
Mario
Carlo
Giovanni
Franco
Lorenzo
Paola
Marco
Rossi
Bianchi
Verdi
Neri
Gialli
Rosati

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

- Q13: Estrarre i nomi ed i cognomi degli impiegati eccetto quelli del dipartimento Amministrazione mantenendo i duplicati.

```

SELECT Nome
FROM Impiegato
WHERE Dipart <> 'Amministrazione'
UNION ALL
SELECT Cognome
FROM Impiegato
WHERE Dipart <> 'Amministrazione'

```

Nome
Carlo
Franco
Carlo
Lorenzo
Marco
Bianchi
Neri
Rossi
Gialli
Franco

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

2.4.8.2 Intersezione

- Q14: Estrarre i cognomi degli impiegati che sono anche nomi

```

SELECT Nome
FROM Impiegato
INTERSECT
SELECT Cognome
FROM Impiegato

```

Nome
Franco

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

2.4.8.3 Differenza

- Q15: Estrarre i nomi degli impiegati che **non** sono cognomi di qualche impiegato

```

SELECT Nome
FROM Impiegato
EXCEPT
SELECT Cognome
FROM Impiegato

```

Nome
Mario
Carlo
Giovanni
Lorenzo
Paola
Marco

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

2.4.9 Operatori aggregati

Una delle più importanti estensioni di SQL rispetto all'algebra relazionale sono gli operatori: **COUNT**, **SUM**, **MAX**, **MIN** e **AVG**

In algebra relazionale tutte le condizioni specificate vengono valutate su una tupla alla volta.

Gli operatori di aggregazione permettono di valutare proprietà che dipendono da insiemi di tuple.

- Prima viene eseguita l'interrogazione considerando solo le clausole **FROM** e **WHERE**, quindi l'operatore di aggregazione viene applicato alla tabella contenente il risultato

2.4.9.1 COUNT

L'operatore COUNT conta il numero di righe di una tabella o il numero di valori di uno o più attributi:

COUNT ((***** | [**DISTINCT** | **ALL**] ListaAttributi))

- *****: conta il numero di righe della tabella
- **DISTINCT ListaAttributi**: conta il numero di valori diversi degli attributi ListaAttributi
- **ALL ListaAttributi**: conta il numero di righe che hanno valori diversi da NULL negli attributi in ListaAttributi
 - Comportamento di default

- Q16: Estrarre il numero di impiegati del dipartimento Produzione

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

- **SELECT COUNT(*)**  **2**
FROM Impiegato
WHERE Dipart = 'Produzione'

- Q17: Estrarre il numero di valori diversi dell'attributo Stipendio fra tutte le righe di Impiegato

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

- **SELECT COUNT(DISTINCT Stipendio)**  **6**
FROM Impiegato

- Q18: Estrarre il numero di righe che possiedono un valore non nullo per l'attributo Nome

• **SELECT COUNT(ALL Nome)**
FROM Impiegato

• **SELECT COUNT(Nome)**
FROM Impiegato



8

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

2.4.9.2 SUM, MAX, MIN, AVG

<SUM | MAX | MIN | AVG > ([DISTINC | ALL] AttrExpr)

- SUM: restituisce la somma dei valori posseduti dall'espressione
- MAX/MIN: restituisce il massimo/minimo dei valori posseduti dall'espressione
- AVG: restituisce la media dei valori

Altre caratteristiche sono:

- SUM e AVG ammettono come argomento solo espressioni che rappresentano valori numerici o intervalli di tempo
- MAX e MIN ammettono come argomento solo espressioni su cui è definito un ordinamento (quindi anche stringhe)
- DISTINCT elimina i duplicati
- ALL trascura solo i valori nulli

SUM

- Q19: Estrarre la somma degli stipendi del dipartimento Amministrazione

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

• **SELECT SUM(Stipendio)**
FROM Impiegato
WHERE Dipart = 'Amministrazione'



125

MAX, MIN, AVG

- Q20: Estrarre gli stipendi minimo, massimo e medio fra quelli di tutti gli impiegati

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

- `SELECT MIN(Stipendio), MAX(Stipendio), AVG(Stipendio)`
`FROM Impiegato`



36, 80, 50.625

- Q21: Estrarre lo stipendio massimo tra quelli degli impiegati che lavorano in un dipartimento con sede a Milano

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

- `SELECT MAX( I.Stipendio )`
`FROM Impiegato as I JOIN Dipartimento as D`
`ON I.Dipart = D.Nome`
`WHERE D.Città = 'Milano'`



80

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano

ATTENZIONE

```
SELECT Cognome, Nome, MAX( I.Stipendio )
FROM Impiegato as I JOIN Dipartimento as D
ON I.Dipart = D.Nome
WHERE D.Città = 'Milano'
```

La seguente interrogazione non è corretta!

Gli operatori aggregati non sono un meccanismo di selezione, ma solo funzioni che restituiscono un valore quando sono applicate ad un insieme.

Cognome e Nome restituiscono un valore per ogni tupla selezionata, mentre MAX() restituisce un valore unico per tutto l'insieme.

Per risolvere questa situazione serve il meccanismo di **raggruppamento** che non vediamo

2.4.10 Interrogazione con Raggruppamento

Gli operatori aggregate possono essere applicati separatamente a sottoinsiemi di righe. La clausola **GROUP BY** permette di specificare come dividere le tabelle in sottoinsiemi. La clausola permette di specificare un insieme di attributi e l'interrogazione raggrupperà le righe che possiedono gli stessi valori per questo insieme di attributi

- Q22: Estrarre la somma degli stipendi di tutti gli impiegati dello stesso dipartimento.

- **SELECT** Dipart, SUM(Stipendio)
FROM Impiegato
GROUP BY Dipart

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

Esecuzione

1. Viene eseguita l'interrogazione come se la clausola **GROUP BY** non esistesse:

```
SELECT Dipart, SUM(Stipendio)
FROM Impiegato
```

Dipart	Stipendio
Amministrazione	45
Produzione	36
Amministrazione	40
Distribuzione	45
Direzione	80
Direzione	73
Amministrazione	40
Produzione	46

2. La tabella ottenuta viene analizzata, dividendo le righe in insiemi caratterizzati dallo stesso valore degli attributi che compaiono nella clausola **GROUP BY**.

Dipart	Stipendio
Amministrazione	45
Amministrazione	40
Amministrazione	40
Produzione	36
Produzione	46
Distribuzione	45
Direzione	80
Direzione	73

3. Dopo che le righe sono state raggruppate in sottoinsiemi, l'operatore aggregato viene applicato separatamente su ogni sottoinsieme.

Dipart	sum(Stipendio)
Amministrazione	125
Produzione	82
Distribuzione	45
Direzione	153

In una interrogazione che fa uso della clausola GROUP BY può comparire come argomento della SELECT solo un sottoinsieme degli attributi usati nella clausola GROUP BY oppure operatori di aggregazione.

• SELECT Ufficio
FROM Impiegato
GROUP BY Dipart



- **ERRORE!!** Ad ogni valore dell'attributo Dipart corrispondono diversi valori dell'attributo Ufficio!
- Ogni sottoinsieme di righe generato dal GROUP BY deve corrispondere ad un'unica riga nella tabella risultato.

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
Paola	Rosati	Amministrazione	75	40
Marco	Franco	Produzione	20	46

È possibile applicare delle selezioni ai gruppi generati tramite la clausola GROUP BY. La clausola **HAVING** permette di specificare delle condizioni che devono essere soddisfatte dai gruppi generati dopo l'esecuzione del raggruppamento tramite GROUP BY.

```
SELECT Dipart, SUM(Stipendio) as SommaStipendi
FROM Impiegato
GROUP BY Dipart
HAVING SUM(Stipendio) > 100
```

Dopo aver eseguito i raggruppamenti, vengono mantenuti solo i gruppi che soddisfano la condizione specificata nella clausola HAVING.

Dipart	Stipendio
Amministrazione	45
Amministrazione	40
Amministrazione	40
Produzione	36
Produzione	46
Distribuzione	45
Direzione	80
Direzione	73

1. Raggruppamento



Dipart	sum(Stipendio)
Amministrazione	125
Produzione	82
Distribuzione	45
Direzione	153

2. Calcolo operatore sui ciascun gruppo



Dipart	SommaStipendi
Amministrazione	125
Direzione	153

3. Selezione con HAVING

OSSERVAZIONE

- La clausola **WHERE** applica delle selezioni su SINGOLE RIGHE, **prima** dell'eventuale raggruppamento tramite GROUP BY.
- La clausola **HAVING** applica delle selezioni su GRUPPI, generati a **seguito** dell'esecuzione del raggruppamento tramite GROUP BY

SQL

```
SELECT Dipart
FROM Impiegato
WHERE Ufficio = 20
GROUP BY Dipart
HAVING AVG(Stipendio) > 25
```

1. WHERE → seleziona le righe che verranno raggruppate
2. GROUP BY → forma i gruppi con le righe selezionate
3. HAVING → seleziona i gruppi che faranno parte del risultato

2.4.11 Interrogazioni nidificate

Una interrogazione nidificata è una interrogazione che contiene al suo interno altre interrogazioni.

La nidificazione può avvenire sia nella clausola **SELECT**, che nella clausola **FROM**, che nella clausola **WHERE**. Il caso tipico è la nidificazione nella clausola WHERE, dove un predicato contiene un confronto tra un valore (espressione su una singola riga) ed il risultato di un'interrogazione SQL.

2.4.11.1 Interrogazioni nidificate: WHERE

Se in un predicato si confronta un attributo o una espressione che produce un valore singolo con il risultato di un'interrogazione, sorge il problema di disomogeneità dei termini del confronto.

I normali operatori di confronto (=, <>, <, >, <=, >=) possono essere estesi con le parole chiave **ANY** e **ALL**.

- ANY**: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **ALMENO** uno degli elementi restituiti dall'interrogazione.
- ALL**: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **TUTTI** gli elementi restituiti dall'interrogazione.

- Q23: Estrarre gli impiegati che lavorano in dipartimenti situati a Firenze

```
SELECT *
FROM Impiegato
WHERE Dipart = ANY (SELECT Nome
FROM Dipartimento
WHERE Città = 'Milano');
```

Dipartimento	Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano	
Produzione	P.le Lavater, 3	Torino	
Distribuzione	Via Segre, 9	Roma	
Direzione	Via Tito Livio, 27	Milano	
Ricerca	Via Venosa, 6	Milano	

→ {Amministrazione, Direzione, Ricerca}

Nome	Cognome	Dipart	Ufficio	Stipendio
Marlo	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	30	36
Giovanni	Venturi	Amministrazione	20	40
Francesco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Giulii	Direzione	7	75
Paola	Rossi	Amministrazione	75	40
Marco	Francesco	Produzione	20	46

- Q23: Estrarre gli impiegati che lavorano in dipartimenti situati a Firenze

```
SELECT *
FROM Impiegato
WHERE Dipart = ANY (SELECT Nome
FROM Dipartimento
WHERE Città = 'Milano');
```

Dipartimento	Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano	
Produzione	P.le Lavater, 3	Torino	
Distribuzione	Via Segre, 9	Roma	
Direzione	Via Tito Livio, 27	Milano	
Ricerca	Via Venosa, 6	Milano	

→ {Amministrazione, Direzione, Ricerca}

Nome	Cognome	Dipart	Ufficio	Stipendio
Marlo	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	30	36
Giovanni	Venturi	Amministrazione	20	40
Francesco	Neri	Distribuzione	16	45
Carlo	Rossi	Direzione	14	80
Lorenzo	Giulii	Direzione	7	75
Paola	Rossi	Amministrazione	75	40
Marco	Francesco	Produzione	20	46

- Q23: Estrarre gli impiegati che lavorano in dipartimenti situati a Firenze

- SELECT *

FROM Impiegato

WHERE

L'interrogazione poteva essere risolta anche in altri modi: JOIN!

FROM Dipartimento
WHERE Città = 'Milano');

Dipartimento

Nome	Indirizzo	Città
Amministrazione	Via Tito Livio, 27	Milano
Produzione	P.le Lavater, 3	Torino
Distribuzione	Via Segre, 9	Roma
Direzione	Via Tito Livio, 27	Milano
Ricerca	Via Venosa, 6	Milano



{Amministrazione, Direzione, Ricerca}

Impiegato

Nome	Cognome	Dipart	Ufficio	Stipendio
Mario	Rossi	Amministrazione	10	45
Carlo	Bianchi	Produzione	20	36
Giovanni	Verdi	Amministrazione	20	40
Franco	Neri	Distribuzione	16	43
Carlo	Rossi	Direzione	14	80
Lorenzo	Gialli	Direzione	7	73
			75	40
			20	46

- Q24: Estrarre i dipartimenti in cui **non** lavorano persone di cognome «Rossi».

```
SELECT Nome
FROM Dipartimento
WHERE Nome <> ALL (SELECT Dipart
FROM Impiegato
WHERE Cognome = 'Rossi');
```

1. L'interrogazione nidificate seleziona i nomi dei dipartimenti in cui Cognome = 'Rossi'

2. La condizione è soddisfatta per tutte quelle righe il cui Nome non fa parte dei nomi restituiti dall'interrogazione nidificata.

In particolare, scrivere:

```
SELECT Nome
FROM Dipartimento
WHERE Nome <> ALL (SELECT Dipart
FROM Impiegato
WHERE Cognome = 'Rossi');
```

è equivalente a scrivere

```
SELECT Nome
FROM Dipartimento
EXCEPT
SELECT Dipart
FROM Impiegato
WHERE Cognome = 'Rossi'
```

Per il controllo di appartenenza o esclusione rispetto ad un insieme si possono usare anche gli operatori IN (= ANY) e NOT IN (<> ALL).

```
SELECT Nome
FROM Dipartimento
WHERE Nome NOT IN (SELECT Dipart
                   FROM Impiegato
                   WHERE Cognome = 'Rossi');
```

Attenzione!

Per le interrogazioni nidificate viste fino ad ora, l'interrogazione può essere eseguita una volta sola ed il risultato ottenuto può essere salvato in una tabella temporanea usata per eseguire ciascun confronto con le righe dalla tabella esterna. Talvolta però l'interrogazione nidificata fa riferimento al contesto dell'interrogazione che la racchiude attraverso l'uso di una variabile (**passaggio di binding**). L'interrogazione nidificata dovrà essere **valutata separatamente per ciascuna riga** prodotta dalla valutazione della query esterna.

2.4.11.2 Interrogazioni nidificate: Binding e EXISTS

L'operatore logico **EXISTS** riceve come parametro un'interrogazione nidificata e restituisce il valore vero solo se l'interrogazione fornisce un risultato non vuoto.

Questo operatore può essere usato in modo significativo solo quando si ha un passaggio di binding tra la query esterna e quella interna

- Q25: Estrarre le persone che hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso)

```
• SELECT *
  FROM Persona P
 WHERE EXISTS (SELECT *
               FROM Persona P1
               WHERE P1.Nome = P.Nome AND
                  P1.Cognome = P.Cognome AND
                  P1.CodFiscale <> P.CodFiscale );
```

Per ogni singola riga esaminata nell'ambito dell'interrogazione esterna, si deve valutare l'interrogazione nidificata.

- Q26: Estrarre le persone che **non** hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso)
- SELECT *
FROM Persona P
WHERE NOT EXISTS (SELECT *
FROM Persona P1
WHERE P1.Nome = P.Nome AND
P1.Cognome = P.Cognome AND
P1.CodFiscale <> P.CodFiscale);
- Q26 alt.: Estrarre le persone che **non** hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso)
- SELECT *
FROM Persona P
WHERE NOT IN (SELECT Nome, Cognome
FROM Persona P1
WHERE P1.CodFiscale <> P.CodFiscale);

2.4.11.3 Interrogazioni nidificate: SELECT

Le interrogazioni nidificate possono essere specificate anche come elemento della clausola SELECT.

- Spesso l'uso di query nidificate nella SELECT sostituiscono l'uso del JOIN.
- Q27: Estrarre per ogni cantante il numero di canzoni di cui è autore.
- SELECT DISTINCT Nome,
NumCanzoni =
(SELECT COUNT(*)
FROM Autore A
WHERE A.Nome = C.Nome)
FROM Cantante C

```
SELECT DISTINCT C.Nome,  
COUNT(*) as NumCanzoni  
FROM Cantante C JOIN Autore A  
ON C.Nome = A.Nome  
GROUP BY C.Nome
```

È necessario che l'interrogazione nidificate restituisca una sola tupla (riferimenti tramite variabile alla interrogazione esterna).

2.4.11.4 Interrogazioni nidificate: FROM

Le interrogazioni nidificate possono essere utilizzate nella clausola FROM come sorgente dati su cui eseguire l'interrogazione.

È possibile creare una catena di interrogazioni, dove ciascuna interrogazione utilizza il risultato della precedente come una tabella di base.

Q28: Estrarre per ogni cantante che è anche autore di canzoni il numero di canzoni di cui è rispettivamente interprete e autore.

- ```
SELECT DISTINCT C.Nome, C.NumCantate, A.NumScritte
FROM
 (SELECT Nome, Count(*) as NumCantate FROM Cantante) C
JOIN
 (SELECT Nome, Count(*) as NumScritte FROM Autore) A
ON C.Nome = A.Nome
```
- Prima vengono eseguite le interrogazioni nella clausola FROM e i risultati di queste vengono usati come normali tabelle.

---

## Strutture fisiche e di accesso ai dati

---

Le basi di dati risiedono nella **memoria secondaria** perchè sono:

- ▷ **grandi**: non possono essere contenute completamente in memoria centrale
- ▷ **persistenti**: hanno un tempo di vita che non è limitato all'esecuzione dei programmi che le usano

### Caratteristiche della memoria secondaria

Le caratteristiche principali della memoria secondaria sono:

- non utilizzabile direttamente dai programmi
- dati organizzati in **blocchi** (grandezza dipende dal sistema operativo)
- operazioni possibili:
  - ◊ lettura intero blocco
  - ◊ scrittura intero blocco
- costo delle operazioni di accesso alla memoria secondaria è maggiore al costo per accedere ai dati in memoria centrale

### 3.1 Gestore del buffer

L'interazione tra memoria secondaria e memoria centrale avviene tramite il **gestore del buffer** tramite il trasferimento di uno o più **blocchi** della memoria secondaria in **pagine** dedicate della memoria centrale (buffer).

#### Precisazione

La dimensione delle pagine in memoria centrale sono solitamente un multiplo della dimensione dei blocchi in memoria secondaria.

Per semplificare, si assume che siano uguali:

$$1 \text{ pagina} \approx 1 \text{ blocco}$$

### 3.1.1 Buffer

Il **buffer** è una zona della memoria condivisa dalle applicazioni e una gestione ottimale del buffer consente di ottenere buone prestazioni nell'accesso ai dati.

Quando uno stesso dato viene utilizzato più volte in tempi ravvicinati il buffer evita l'accesso alla memoria secondaria.

#### Compito del gestore del buffer

Il gestore del buffer si occupa di caricare e salvare i dati dalla memoria secondaria alla memoria centrale.

### 3.1.2 Politica di gestione della memoria

L'obiettivo del gestore del buffer è di **ottimizzare gli accessi** attraverso le politiche di gestione della memoria:

#### ◇ lettura di un blocco:

- ▷ blocco presente in una pagina del buffer
  - ⇒ non viene eseguita la lettura in memoria secondaria
  - ⇒ viene restituito un puntatore alla pagina del buffer
- ▷ blocco non presente in una pagina del buffer
  - ⇒ viene cercata una pagina libera
  - ⇒ viene caricato il blocco nella pagina
  - ⇒ viene restituito il puntatore alla pagina del buffer

#### ◇ scrittura di un blocco: se viene effettuata una richiesta di scrittura di un blocco già caricato in una pagina del buffer, il gestore del buffer può decidere di **differire la scrittura** su memoria secondaria in un secondo momento

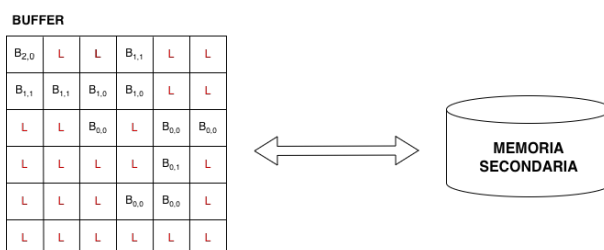
Per poter aumentare la velocità di accesso ai dati, le politiche di lettura e scrittura si basano sul **principio di località**.

#### Definizione: principio di località

Il **principio di località** si basa sul fatto che i dati referenziati di recente hanno maggiore probabilità di essere referenziati nuovamente in futuro.

### 3.1.3 Gestione delle pagine

Il contenuto del buffer lo possiamo immaginare come una matrice.



Per ogni pagina del buffer si memorizza:

- ▷ blocco B (L se libero) indicando il file e il numero di blocco (offset)
- ▷ variabili di stato ( $B_{i,j}$ ):
  - contatore i: indica il numero di transazioni
  - bit di stato j: indica se la pagina è stata modificata (1) o no (0)

### 3.1.4 Primitive per la gestione del buffer

#### 3.1.4.1 Primitiva fix

La **primitiva fix** viene utilizzata dalle transazioni per richiedere l'accesso ad un blocco e restituisce un puntatore alla pagina contenente il blocco richiesto.

Le azioni della primitiva sono:

- ▷ **ricerca del blocco:**
  - ◊ blocco viene trovato
    - ⇒ viene restituito un puntatore alla pagina contenente il blocco
  - ◊ blocco non viene trovato:
    - viene scelta una pagina libera
      - ⇒ viene salvata in memoria secondaria (primitiva flush)
      - ⇒ viene caricato il blocco in P aggiornando file e numero di blocco
    - non ci sono pagine libere
      - ⇒ viene rubata una pagina a un'altra transazione (primitiva flush)
      - ⇒ viene sospesa la transazione inserendola in una coda di attesa
      - ⇒ se si libera una pagina il gestore si comporterà come al punto precedente

⇒ **effetto della modifica:** quando una transazione accede a una pagina per la prima volta il contatore si incrementa

#### 3.1.4.2 Primitiva unfix

La **primitiva unfix** viene usata dalle transazioni per indicare che la transazione ha terminato di usare il blocco.

⇒ **effetto della modifica:** decremento del contatore i

#### 3.1.4.3 Primitiva setDirty

La **primitiva setDirty** viene utilizzata dalle transazioni per indicare al gestore del buffer che il blocco della pagina è stato modificato.

⇒ **effetto della modifica:** bit di stato J passa da 0 a 1

#### 3.1.4.4 Primitiva force

La **primitiva force** viene utilizzata per salvare in memoria secondaria in **modo sincrono** il blocco contenuto nella pagina.

Questa primitiva viene usata dal gestore dell'affidabilità.



⇒ **effetto della modifica**: viene salvato in memoria secondaria il blocco e il bit di stato J posto a 0

### 3.1.4.5 Primitiva flush

La **primitiva flush** viene utilizzata per salvare i blocchi in memoria secondaria in **modo asincrono**.

Questa primitiva viene usata dal gestore del buffer.

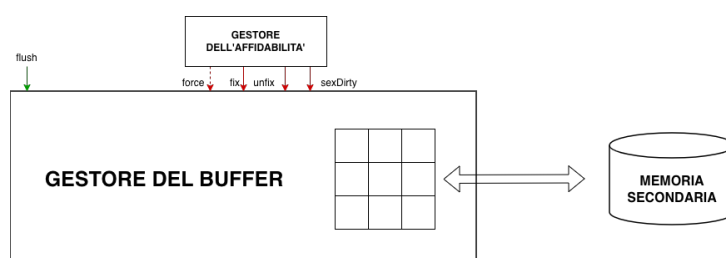
⇒ **effetto della modifica**: viene liberata la pagina "dirty" e il bit di stato J viene posto a 0

## 3.2 Gestore dell'affidabilità

Il gestore del buffer differisce le operazioni di scrittura e questo potrebbe rappresentare un pericolo.

Il **gestore dell'affidabilità** si occupa di garantire:

- ▷ **atomicità**: transazioni non devono essere incomplete
- ▷ **persistenza**: effetti di ciascuna transazione non conclusa con un commit devono essere mantenuti in modo permanente



### 3.2.1 File di LOG

Il funzionamento del gestore dell'affidabilità si basa sull'utilizzo del **file di LOG**.

#### Definizione: file di LOG

Il **file di LOG** è un archivio persistente su cui vengono registrate tutte le operazioni eseguite dal DBMS.

Per il corretto funzionamento, il gestore dell'affidabilità deve disporre di un dispositivo di una **memoria stabile** (resistente ai guasti).

Il file di LOG viene memorizzato in memoria stabile e al suo interno vengono memorizzati:

- ▷ **record di transizione**: record che descrivono informazioni sulle transazioni
  - ◊ **B(T)**: **begin** della transazione
  - ◊ **C(T)**: **commit** della transazione
  - ◊ **A(T)**: **abort** della transazione
  - ◊ **I(T,O,AS)**: **insert** di un oggetto da parte di una transazione T con il suo after state AS (stato dell'oggetto dopo l'operazione)

- ◇ **D(T, O, BS)**: **delete** di un oggetto O da parte della transizione T con il suo before state BS (stato dell'oggetto prima dell'operazione)
- ◇ **U(T, O, BS, AS)**: **update** di un oggetto O da parte della transizione T con il suo before state BS e il suo after state AS
- ▷ **record di sistema**: record che descrivono informazioni sulle operazioni eseguite dal DBMS
  - **DUMP**: copia completa e consistente della base di dati.
  - **CK( $T_1, \dots, T_n$ )**: indica che all'esecuzione del checkpoint, le transazioni attive erano  $T_1, \dots, T_n$ 
    - \* elenco delle transazioni che non hanno ancora espresso la loro scelta relativamente all'esito finale
    - \* operazione svolta periodicamente affinché i dati delle transazioni che hanno eseguito il commit siano registrati in modo permanente
    - \* il protocollo di checkpoint prevede i seguenti passi:
      1. sospensione delle operazioni di scrittura, commit e abort delle transazioni
      2. esecuzione della primitiva force sulle pagine "dirty" di transazioni che hanno eseguito il commit
      3. scrittura sincrona (primitiva force) sul file di LOG del record di checkpoint con gli identificatori delle transazioni attive
      4. ripresa delle operazioni di scrittura, commit e abort delle transazioni

I record di transazione salvati nel LOG consentono di eseguire in caso di ripristino le seguenti operazioni:

- ▷ **UNDO**: viene usato per disfare un'azione su un oggetto O è sufficiente ricopiare in O il valore di BS.  
L'insert/delete viene disfatto cancellando/inserendo O
- ▷ **REDO**: serve per rifare un'azione su un oggetto O è sufficiente ricopiare in O il valore AS.  
L'insert/delete viene rifatto inserendo/cancellando O

Gli inserimenti controllano sempre l'esistenza di O prima di eseguire l'operazione.

Per queste operazioni vale la **proprietà di idempotenza**.

#### Definizione: proprietà di idempotenza

$$\begin{aligned}\text{UNDO}(\text{UNDO}(A)) &= \text{UNDO}(A) \\ \text{REDO}(\text{REDO}(A)) &= \text{REDO}(A)\end{aligned}$$

### 3.2.2 Regole di scrittura sul file di log

Il gestore dell'affidabilità deve rispettare le seguenti regole di scrittura sul file di LOG:

- ⇒ **regola Write Ahead Logging**: record di LOG devono essere scritti sul LOG precedente all'esecuzione delle corrispondenti operazioni sulla base di dati (garantisce la possibilità di fare sempre UNDO)
- ⇒ **regola di commit-precedenza**: record di LOG devono essere scritti sul LOG prima dell'esecuzione del commit della transazione (garantisce la possibilità di fare sempre REDO)

Queste regole consentono di salvare i blocchi delle pagine “dirty” in modo totalmente asincrono rispetto al commit delle transazioni.

### 3.2.3 Guasti

Una transazione  $T$  sceglie in modo atomico l'esito di commit nel momento in cui scrive nel file di log in modo sincrono (primitiva *force*) il suo record di commit  $C(T)$ .

⇒ se avviene un guasto prima della scrittura del record di commit → **UNDO**

⇒ se avviene un guasto dopo la scrittura del record di commit → **REDO**

Esistono due tipologie di guasto:

- ▷ **guasto di sistema**: perdita del conte della memoria centrale.

Viene risolto con la tecnica della **ripresa a caldo**:

1. accede al file di LOG e si ripercorre all'indietro il LOG fino al più recente record di checkpoint
2. si inizializza:
  - ◊ insieme UNDO con tutte le transazioni da disfare (presenti nel checkpoint)
  - ◊ insieme REDO con tutte le transazioni da rifare ( $\emptyset$ )
3. si ripercorre in avanti il LOG
  - per ogni record  $B(T)$  incontrato si aggiunge  $T$  all'insieme UNDO
  - per ogni record  $C(T)$  incontrato si sposta  $T$  dall'insieme UNDO all'insieme REDO
4. si ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni UNDO risalendo fino alla prima azione della transazione più vecchia
5. si ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

- ▷ **guasto di dispositivo**: perdita di parte o tutto il contenuto della base di dati in memoria secondaria.

Viene risolto con la tecnica della **ripresa a freddo**:

1. si accede al DUMP della base di dati e si ricopia selettivamente la parte deteriorata della base di dati
2. si accede al LOG risalendo al record di DUMP
3. si ripercorre in avanti il LOG rieseguendo tutte le operazioni relative alla parte deteriorata comprese le azioni di commit e abort
4. si applica una ripresa a caldo

#### Esempio: file di LOG

Consideriamo il seguente file di LOG:

$B(T_1), B(T_2), U(T_2, O_1, B_1, A_1), I(T_1, O_2, A_2), B(T_3), C(T_1), B(T_4),$   
 $U(T_3, O_2, B_3, A_3), U(T_4, O_3, B_4, A_4), CK(T_2, T_3, T_4), C(T_4), B(T_5),$   
 $U(T_3, O_3, B_5, A_5), U(T_5, O_4, B_6, A_6), D(T_3, O_5, B_7), A(T_3), C(T_5),$   
 $I(T_2, O_6, A_8)$  **guasto**

**Che passi svolge la ripresa a caldo?**

1. risale il LOG fino all'ultimo record di checkpoint  $CK(T_2, T_3, T_4)$
2. inizializza UNDO e REDO

- ▷ UNDO =  $\{T_2, T_3, T_4\}$  (inizializzato con le transazioni del checkpoint)
- ▷ REDO =  $\emptyset$
- 3. ripercorre in avanti il LOG dal CheckPoint fino al guasto e si aggiungono o si spostano le transazioni tra UNDO e REDO
  - ▷ UNDO =  $\{T_2, T_3, \cancel{T_4}, \cancel{T_5}\}$  (inizializzato con le transazioni del checkpoint)
  - ▷ REDO =  $\{T_4, T_5\}$
- 4. ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni UNDO risalendo fino alla prima azione della transazione più vecchia

DELETE( $O_6$ ) (disfare insert)

$O_5 := B_7$  (disfare delete)

$O_3 := B_5$  (disfare update)

$O_2 := B_3$  (disfare update)

$O_1 := B_1$  (disfare update)

- 5. ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

$O_3 := A_4$  (rifare update)

$O_4 := A_6$  (rifare update)

### Esempio: file di LOG

Consideriamo il seguente file di LOG:

B( $T_1$ ), B( $T_2$ ), B( $T_3$ ), I( $T_1, O_1, A_1$ ), D( $T_2, O_2, B_2$ ), B( $T_4$ ),  
 U( $T_4, O_3, B_3, A_3$ ), U( $T_1, O_4, B_4, A_4$ ), C( $T_2$ ), CK( $T_1, T_3, T_4$ ), B( $T_5$ ), B( $T_6$ ),  
 U( $T_5, O_5, B_5, A_5$ ), A( $T_3$ ), CK( $T_1, T_4, T_5, T_6$ ), B( $T_7$ ), C( $T_4$ ),  
 U( $T_7, O_6, B_6, A_6$ ), U( $T_6, O_3, B_7, A_7$ ), B( $T_8$ ), A( $T_7$ ) **guasto**

### Che passi svolge la ripresa a caldo?

1. risale il LOG fino all'ultimo record di checkpoint CK( $T_1, T_4, T_5, T_6$ )
2. inizializza UNDO e REDO
  - ▷ UNDO =  $\{T_1, T_4, T_5, T_6\}$  (inizializzato con le transazioni del checkpoint)
  - ▷ REDO =  $\emptyset$
3. ripercorre in avanti il LOG dal CheckPoint fino al guasto e si aggiungono o si spostano le transazioni tra UNDO e REDO
  - ▷ UNDO =  $\{T_1, \cancel{T_4}, T_5, t_6, t_7, t_8\}$  (inizializzato con le transazioni del checkpoint)
  - ▷ REDO =  $\{T_4\}$
4. ripercorre all'indietro il LOG disfacendo le operazioni eseguite dalle transazioni

UNDO risalendo fino alla prima azione della transazione più vecchia

$O_3 := B_7$  (disfare update)

$O_6 := B_6$  (disfare update)

$O_5 := B_5$  (disfare update)

$O_4 := B_4$  (disfare update)

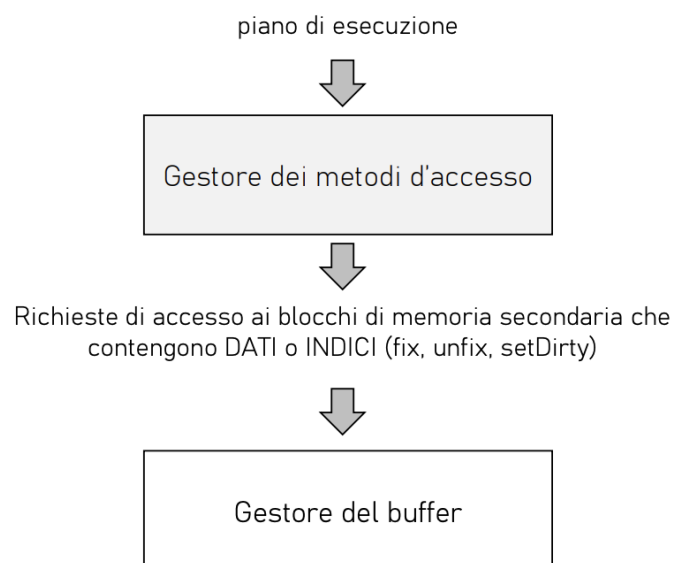
DELETE( $O_1$ ) (disfare insert)

5. ripercorre in avanti il LOG per rifare le operazioni eseguite dalle transazioni REDO

$O_3 := A_3$  (rifare update)

### 3.3 Gestore dei metodi di accesso

Il gestore dei metodi di accesso esegue il piano di esecuzione prodotto dall'ottimizzatore e produce sequenze richieste di accessi ai blocchi della base di dati presenti in memoria secondaria. Le richieste vengono inviate al gestore del buffer che si occupa di caricare i blocchi necessari in pagine di memoria centrale.



**Figura 3.1:** gestore dei meotodi di accesso

#### 3.3.1 Metodi di accesso

I metodi di accesso sono moduli software (es. scansione sequenziale, accesso via indice, ordinamento, varie implementazioni del join) che implementano gli algoritmi di accesso e manipolazione dei dati organizzati in specifiche strutture fisiche.

Ogni metodo di accesso conosce:

- ◇ **organizzazione in tuple** (o dei record di indice) nei blocchi dati (indice) salvati in memoria secondaria
- ◇ **organizzazione fisica interna delle pagine** sia quando contengono dati, sia quando contengono strutture fisiche di accesso (indici)

### 3.3.2 Organizzazione di una pagina

All'interno di una pagina sono presenti:

- ▷ **informazioni utili**: dati veri e propri (tuple della tabella)
- ▷ **informazioni di controllo**: consentono di accedere alle informazioni utili (dizionario, bit di parità, altre informazioni del file system o della specifica struttura fisica)

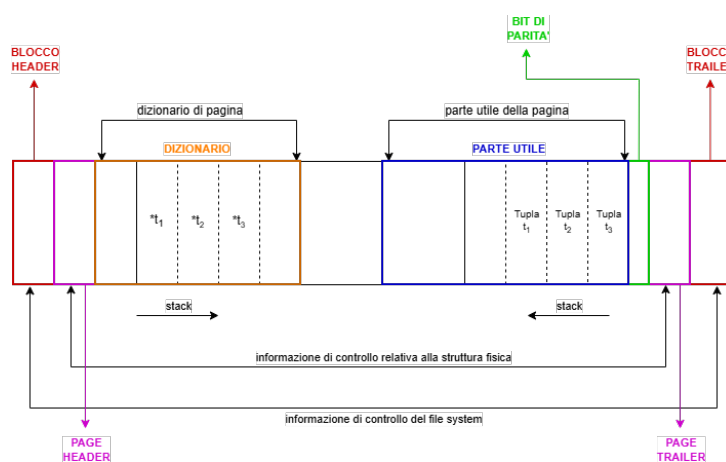


Figura 3.2: organizzazione di una pagina

- ⇒ **blocco header/trailer**: contengono informazioni di controllo utilizzate dal file system
- ⇒ **pagina header/trailer**: contengono le informazioni di controllo relative alla specifica struttura fisica
- ⇒ **dizionario di pagina**: contiene i puntatori a ciascun dato utile elementare contenuto nella pagina
- ⇒ **bit di parità**: verifica che l'informazione contenuta nella pagina sia valida
- ⇒ **parte utile**: contiene i dati

#### N.B.

Dizionari di pagina e parte utile crescono come stack contrapposti, lasciando libera la parte centrale in uno spazio contiguo.

In caso di **tuple di lunghezza fissa**, il dizionario non è necessario: si memorizza la dimensione delle tuple e l'offset del punto iniziale.

In caso di **tuple di lunghezza variabile**, il dizionario memorizza l'offset di ogni tupla presente nel blocco e di ogni attributo di ogni tupla.

Molti gestori delle pagine non consentono la separazione di una tupla su più pagine:

$$\text{dimensione massima tupla} = \text{dimensione massima area disponibile su un blocco}$$

Alcuni gestori delle pagine consentono di distribuire una tupla su più pagine, altrimenti va gestito il caso di tuple memorizzato su più pagine.

### 3.3.3 Operazioni sulle pagine

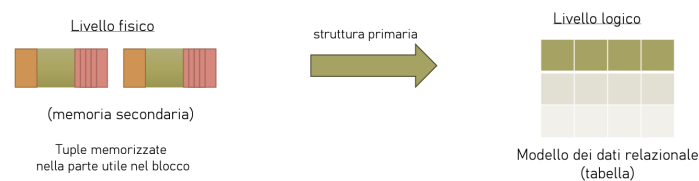
Le principali operazioni che si possono eseguire sulle pagine sono:

- ▷ **inserimento/aggiornamento di una tupla:**
  - ◇ esiste spazio contiguo sufficiente: Inserimento
  - ◇ non esiste spazio contiguo sufficiente, ma spazio sufficiente: riorganizzazione dello spazio + inserimento
  - ◇ non esiste spazio sufficiente: interazione con il file system per allocare nuovi blocchi per il file
- ▷ **cancellazione contenuto della pagina**
- ▷ **accesso a una tupla:** avviene tramite il valore della chiave o l'offset nel dizionario
- ▷ **accesso a un attributo di una tupla:** identificato in base all'offset e alla lunghezza del campo dopo aver identificato la tupla tramite la chiave o il suo offset

### 3.3.4 Strutture primarie per l'organizzazione dei file

La struttura primaria di un file stabilisce il criterio di disposizione delle tuple all'interno del file presente in memoria di massa:

- ▷ **sequenziale:** colloca le tuple in maniera consecutiva sulla base di un criterio specifico (es. ordine di inserimento)
- ▷ **ad accesso calcolato:** colloca le tuple in posizioni determinate sulla base dell'esecuzione di un algoritmo di hash
- ▷ **ad albero:** utilizzate come strutture secondarie (indici)



**Figura 3.3:** strutture primarie per l'organizzazione dei file

#### 3.3.4.1 Struttura sequenziale

Un file è costituito da blocchi "logicamente" consecutivi e le tuple vengono inserite nei blocchi rispettando una sequenza.

##### 3.3.4.1.1 Struttura sequenziale disordinata

Le tuple sono disposte in base al loro ordine di inserimento, senza alcun criterio specifico. Questa è la struttura più semplice e più diffusa.

⇒ **inserimento:**

- ◇ operazione molto efficiente

- ◇ sufficiente mantenere un riferimento all'ultimo blocco
- ◇ richiede solo un accesso in memoria secondaria
- ◇ maggiore costo in caso di verifica dei vincoli di integrità

⇒ **ricerca**

- ◇ operazione non efficiente
- ◇ richiede ogni volta una scansione sequenziale che consideri tutti i record, con un costo lineare nel numero di blocchi
- ◇ ricerca può essere fatta usando strutture secondarie (indici)

⇒ **eliminazione**

- ◇ facile una volta individuate le tuple coinvolte
- ◇ si marcano le tuple come "cancellate" senza riorganizzazione locale

⇒ **modifica**

- ◇ facile una volta individuate le tuple coinvolte
- ◇ si procede in loco se possibile, cioè quando non c'è stata nessuna modifica alla dimensione della tupla
- ◇ si procede con una cancellazione e un inserimento in fondo al file

Eliminazioni e modifiche possono portare ad un uso non ottimale dello spazio siccome si devono effettuare riorganizzazioni periodiche.

#### 3.3.4.1.2 Struttura sequenziale ad array

Le tuple sono disposte come in un array, e la loro posizione dipende dal valore assunto in ciascuna tupla da un campo detto **indice**.

Questa struttura è utilizzabile soltanto quando le tuple di una tabella hanno una dimensione fissa, infatti non è quasi mai usata nei DBMS reali perchè difficilmente sono soddisfatte le condizioni per la sua applicazione.

Ad ogni file viene assegnato un numero  $n$  di blocchi contigui e ciascun blocco è dotato di un numero  $m$  di posizioni disponibili per le tuple (quindi è un array bidimensionale  $n \times m$ ).

Ogni tupla è dotata di un valore numerico  $i$  che rappresenta l'indice della tupla in  $i$ -esima posizione nell'array.

⇒ **inserimento**

- Iniziale: l'indice  $i$  viene ottenuto incrementando un contatore
- Successivo: in una posizione libera oppure alla fine del file

⇒ **cancellazione**: creano delle posizioni libere

#### 3.3.4.1.3 Struttura sequenziale ordinata

È un file sequenziale dove le tuple sono ordinate secondo una chiave di ordinamento (diversa dalla chiave primaria).

Questa struttura rende efficienti le operazioni che hanno bisogno dell'ordinamento utilizzato, ad esempio:

- elenco ordinato
- range query



- operazioni aggregate

In caso di aggiornamento è necessario mantenere l'ordinamento, quindi servono periodiche riorganizzazioni.

⇒ **inserimento**

1. individuare il blocco B che contiene la tupla che precede, nell'ordine della chiave, la tupla da inserire
2. se B contiene spazio sufficiente per la nuova tupla allora si inserisce la tupla in B
3. altrimenti si aggiunge un nuovo blocco, detto overflow page, alla struttura e si inserisce la tupla nel nuovo blocco e si aggiusta la catena di puntatori

⇒ **scansione sequenziale ordinata secondo la chiave**: è sufficiente seguire i puntatori

⇒ **cancellazione di una tupla**

1. individuare il blocco B che contiene la tupla da cancellare
2. Cancellare la tupla da B
3. aggiustare la catena di puntatori

⇒ **riorganizzazione**: si assegnano le tuple ai blocchi in base ad opportuni coefficienti di riempimento, riaggiustando i puntatori

#### Esempio: struttura sequenziale

Il seguente esempio mostra una struttura sequenziale ordinata secondo la chiave Filiale.

chiave ordinamento



|          | Filiale | Conto | Cliente | Saldo |                                                                                       |
|----------|---------|-------|---------|-------|---------------------------------------------------------------------------------------|
| Blocco 1 | A       | 102   | Rossi   | 1000  |  |
|          | B       | 110   | Rossi   | 3020  |                                                                                       |
|          | B       | 198   | Bianchi | 500   |                                                                                       |
| Blocco 2 | E       | 17    | Neri    | 345   |                                                                                       |
|          | E       | 102   | Verdi   | 1200  |                                                                                       |
|          | E       | 113   | Bianchi | 200   |                                                                                       |
|          | H       | 53    | Neri    | 120   |                                                                                       |
|          | F       | 78    | Verdi   | 3400  |                                                                                       |

#### 3.3.4.1.4 Indice

Per aumentare le prestazioni degli accessi alle tuple memorizzate nelle strutture fisiche (file sequenziale), si introducono strutture ausiliarie dette strutture di accesso ai dati, o indici.

Queste strutture velocizzano l'accesso casuale via chiave di ricerca, cioè un insieme di attributi utilizzati nell'indice della ricerca.

Ci sono due tipi di indici su file sequenziali:

- ▷ **indice primario**: la chiave di ordinamento del file sequenziale coincide con la chiave di ricerca dell'indice.

Ogni record dell'indice primario contiene una coppia  $\langle v_i, p_i \rangle$ , dove:

- $v_i$  è il valore della chiave di ricerca
- $p_i$  è il puntatore al primo record nel file sequenziale con chiave  $v_i$

Esistono due varianti dell'indice primario:

- **indice denso**: per ogni occorrenza della chiave presente nel file esiste un corrispondente record nell'indice
- **indice sparso**: solo per alcune occorrenze della chiave presenti nel file esiste un corrispondente record nell'indice, tipicamente una per blocco

Le operazioni che si possono eseguire su un indice primario sono:

- **ricerca** di una tupla con chiave di ricerca  $K$ 
  - \* Se l'indice è denso  $K$  è presente nell'indice
    1. Scansione sequenziale dell'indice per trovare il record  $(K, pk)$
    2. Accesso al file attraverso il puntatore  $pk$
 Il costo è 1 accesso indice + 1 accesso blocco dati
  - \* Se l'indice è sparso  $K$  potrebbe non essere presente nell'indice
    1. Scansione sequenziale dell'indice fino al record  $(K'', pk)$ , dove  $K''$  è il valore più grande che sia minore o uguale a  $K$
    2. Accesso al file attraverso il puntatore  $pk$  e scansione del file (blocco corrente) per trovare le tuple con chiave  $K$
 Il costo è 1 accesso indice + 1 accesso blocco dati

- **Inserimento** di un record nell'indice. Si effettuano gli stessi passi dell'inserimento in un file sequenziale (3.3.5), solo che nel blocco di memoria secondaria al posto di tuple ci sono record di indice.

- \* Se l'indice è denso l'inserimento nell'indice avviene solo se la tupla inserita nel file ha un valore di chiave  $K$  che non è già presente
- \* Se l'indice è sparso l'inserimento avviene solo quando, per effetto dell'inserimento di una nuova tupla, si aggiunge un blocco dati alla struttura; in tutti gli altri casi l'indice rimane invariato

Gli indici sono costosi, quindi bisogna fare un compromesso sul numero di indici da mantenere e il numero di attributi ad accesso sequenziale.

- **Cancellazione** di un record nell'indice. Si effettuano gli stessi passi della cancellazione in un file sequenziale (3.3.5)
  - \* Se l'indice è denso la cancellazione nell'indice avviene solo se la tupla cancellata nel file è l'ultima tupla con valore di chiave  $K$
  - \* Se l'indice è sparso la cancellazione nell'indice avviene solo quando  $K$  è presente nell'indice e il corrispondente blocco viene eliminato; altrimenti, se il blocco sopravvive, va sostituito  $K$  nel record dell'indice con il primo valore  $K''$  presente nel blocco.

▷ **indice secondario**: La chiave di ordinamento e la chiave di ricerca sono diverse.

Ogni record dell'indice secondario contiene una coppia  $\langle vi, pi \rangle$ , dove:

- $vi$  è il valore della chiave di ricerca
- $pi$  è il puntatore al bucket di puntatori che individuano nel file sequenziale tutte le tuple con chiave  $vi$

Gli indici secondari sono sempre densi perchè non si può sfruttare l'ordinamento delle tuple.

Le operazioni che si possono eseguire su un indice secondario sono:

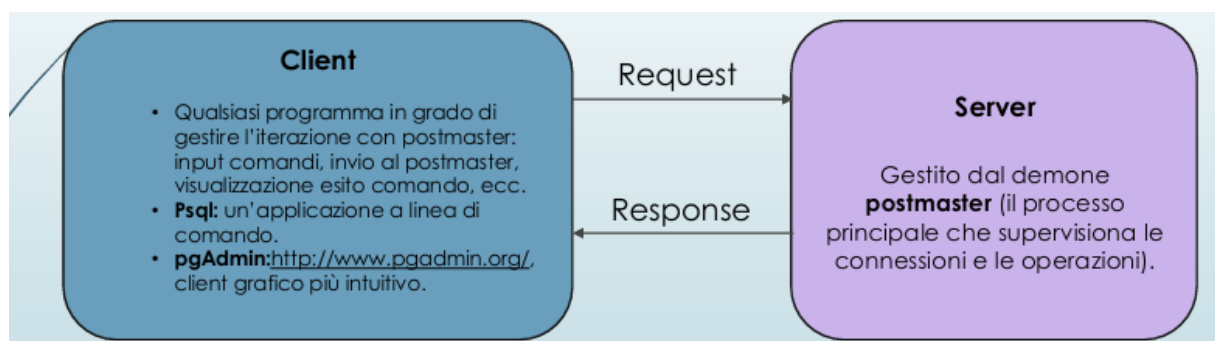
- Ricerca di una tupla con chiave di ricerca  $K$ 
  - \* Scansione sequenziale dell'indice per trovare il record  $(K, pk)$
  - \* Accesso al bucket  $B$  di puntatori attraverso il puntatore  $pk$
  - \* Accesso al file attraverso i puntatori del bucket  $B$
 Il costo è 1 accesso indice + 1 accesso al bucket +  $n$  accessi alle pagine dati dove  $n$  è il numero di puntatori nel bucket  $B$
- Inserimento e cancellazione: si effettuano gli stessi passi dell'inserimento e della can-

---

cellazione in un indice primario denso, con in più l'aggiornamento dei bucket.

## Laboratorio

PostgreSQL è un Relational Data Base Management System (RDBMS o DBMS) con alcune funzionalità orientate agli oggetti. L'interazione tra un utente (programmatore/utente finale) e le basi di dati gestite da PostgreSQL avviene secondo il modello client-server.



### 4.1 Creazione di una tabella

Struttura base

```
CREATE TABLE tabella (
 attributo dominio [valoreDefault] { vincolo }
 {, attributo dominio [valoreDefault] { vincolo } }
 {, vincoloTabella }
);
```

Costrutti inseriti:

- [] indicano parti opzionali / facoltative (il termine può non comparire o comparire una sola volta) Indica che quell'elemento può apparire zero o una volta. Sta al programmatore decidere se inserirlo o meno a seconda di ciò che si vuole ottenere)
- [ valoreDefault ]: Significa che quando definisco l'attributo posso decidere se attribuirgli un valore predefinito
- { } Tutto ciò che è racchiuso tra parentesi graffe rappresenta un elemento che può essere ripetuto zero, una o più volte.

- { vincolo }: Significa che a una singola colonna puoi applicare nessun vincolo, un solo vincolo, oppure una serie di vincoli uno dopo l'altro (es. NOT NULL, UNIQUE)

### Ecco un esempio

```
CREATE TABLE Impiegato (
 Matricola CHARACTER(6) PRIMARY KEY,
 Nome VARCHAR(20) NOT NULL,
 Cognome VARCHAR(20) NOT NULL,
 Dipart VARCHAR(15),
 Stipendio NUMERIC(9) DEFAULT 0,
 FOREIGN KEY(Dipart)
 REFERENCES Dipartimento(NomeDip), /*vincolo di tabella*/
 UNIQUE (Cognome, Nome) /*vincolo di tabella*/
)
```

#### 4.1.1 Identificatori

Sono stringhe che iniziano con una lettera (UTF-8) o un underscore (\_) e che possono contenere anche cifre (0-9). Gli identificatori SQL **NON** sono sensibili al minuscolo/maiuscolo:

- idPersona e idpersona rappresentano un solo identificatore.

#### 4.1.2 Domini

I domini elementari (predefiniti) sono:

- **caratteri**: singoli caratteri o stringhe anche di lunghezza variabile
- **bit**: bit singoli (booleani) o stringhe di bit
- **numerici esatti e approssimati**
- **istanti temporali**
- **intervalli temporali**

Oppure si possono avere domini definiti dall'utente

##### 4.1.2.1 Tipo carattere

Permette di rappresentare singoli caratteri oppure stringhe. Un carattere o stringa di caratteri sono rappresentati tra apici ('): 'a', 'Questa è una stringa'.

La lunghezza delle stringhe può essere fissa o variabile: **CHARACTER [VARYING][(lunghezza)]**

- **CHARACTER**: carattere singolo
- **CHARACTER(20)**: stringa di **lunghezza fissa**. Se si assegna una stringa di lunghezza inferiore a 20, la stringa viene riempita di spazi fino a rendere la stringa lunga 20.
- **CHARACTER VARYING(20)**: stringa di **lunghezza variabile** (max 20).
- **TEXT**: stringa di lunghezza variabile senza limite fissato. Questa è un'estensione PostgreSQL

### 4.1.2.2 Tipo boolean

Permette di rappresentare i valori booleani TRUE/FALSE, rappresentati anche come t/f, 1/0, yes/NO, più lo stato unknown (valore nullo), rappresentato come NULL

### 4.1.2.3 Tipo numerici esatti

Permettono di rappresentare valori esatti: interi o con una parte decimale di lunghezza prefissata

- **Valori interi:**
  - SMALLINT: valori interi (2 bytes: da -32.768 (215) a +32.767 (215-1)).
  - INTEGER: valori interi (4 bytes: da -2147483648 (231) to +2147483647 (231-1) ).
- **Valori decimali:**
  - NUMERIC [(precisione [, scala])]: precisione è il numero totale di cifre significative (cioè di cifre a sinistra e a destra della virgola), scala è il numero di cifre dopo la virgola (vuoto = 0).
  - DECIMAL [(precisione [, scala])]: equivalente al precedente.

#### Esempio:

NUMERIC(5,2) permette di rappresentare valori come 100.01, 100.999 (arrotondato a 101.00), ma non valori come 1000.01

### 4.1.2.4 Tipo numerici approssimati

Permettono di rappresentare valori in virgola mobile:

- **REAL:** una precisione di 6 cifre decimali.
- **DOUBLE PRECISION:** una precisione di 15 cifre decimali.

Ciascun numero è rappresentato da una coppia di valori: la mantissa e l'esponente.

- La mantissa è un numero frazionario, l'esponente è un numero intero
- Il valore si ottiene moltiplicando la mantissa per la potenza di 10 con grado pari all'esponente

Il numero 0.17E16 corrisponde a  $1,7 \times 10^{15}$  Il numero -0.4E-6 corrisponde a  $-4 \times 10^{-7}$

### 4.1.2.5 Istanti temporali

Permettono di rappresentare istanti di tempo.

- **DATE:** istanti del tipo (anno, mese, giorno).
  - Un valore DATE si rappresenta tra apici (') e lo standard ISO prescrive il formato YYYY-MM-DD.
- **TIME [(precisione)][WITH TIME ZONE]:** istanti del tipo (hour, minute, second)
  - precisione: numero cifre decimali per rappresentare frazioni del secondo;
  - WITH TIME ZONE: permette di specificare anche [+ -]hour:minute, che rappresentano la differenza con l'ora di Greenwich, o nomeFusoOrario.
- **TIMESTAMP [(precisione)][WITH TIME ZONE]:** date + time.

### 4.1.2.6 Intervalli temporali

INTERVAL PrimaUnitàDiTempo[TO UltimaUnitàDiTempo] PrimaUnitàDiTempoe UltimaUnitàDiTempo definiscono le unità di misura che devono essere utilizzate, dalla più precisa alla meno precisa. L'insieme delle unità di misura è diviso in due insiemi distinti:

- Year-Month
- Day-Hour-Minute-Second

### 4.1.2.7 Domini definiti dall'utente

```
CREATE DOMAIN nome AS tipoBase [valoreDefault][vincolo]
```

**CREATE DOMAIN:** definisce un dominio, utilizzabile in definizioni di relazioni, anche con vincoli e valori di default. Il dominio può essere definito a partire da un dominio elementare oppure da un altro dominio definito dall'utente.

```
CREATE DOMAIN giorniSettimana AS CHARACTER (3)
CHECK (VALUE IN ('LUN', 'MAR', 'MER', 'GIO', 'VEN', 'SAB', 'DOM'));
```

```
CREATE DOMAIN Voto AS SMALLINT DEFAULT NULL
CHECK (value >=18 AND value <= 30)
```

### 4.1.3 Vincoli intrarelazionali

- **NOT NULL:** il valore nullo non è ammesso per l'attributo, quindi il valore deve sempre essere specificato, o si assegna un valore di default
- **UNIQUE:** definisce (super)chiavi

```
Nome VARCHAR(20),
Cognome VARCHAR(20),
UNIQUE(Nome, Cognome)
```

```
/* impone che non ci siano due righe che abbiano uguale sia il nome che
```

```
Nome VARCHAR(20) UNIQUE,
Cognome VARCHAR(20) UNIQUE
```

```
/* impone che non ci siano due righe che abbiano lo stesso nome o lo stesso
```

- **PRIMARY KEY:** chiave primaria (una sola, implica NOT NULL)

```
matricola CHAR (6) PRIMARY KEY;
/* se unico componente della chiave primaria*/
```

```
nome VARCHAR(20),
cognome VARCHAR(20),
PRIMARY KEY(nome, cognome)
```

```
/* Se la chiave primaria ha piu attributi */
```

- **CHECK:** specifica un vincolo generico che deve soddisfare le tuple della tabella. Viene soddisfatto se la sua espressione è vera o nulla.

```
CREATE TABLE Impiegato (
 matricola CHAR(6) PRIMARY KEY,
 nome VARCHAR(20) NOT NULL,
 cognome VARCHAR(20) NOT NULL,
 qualifica VARCHAR(20),
 stipendio NUMERIC(8,2) DEFAULT 500.00 NOT NULL
 CHECK(stipendio >= 0.0), /* check di attributo */
 UNIQUE(cognome, nome),
 CHECK(nome <> cognome) /* check di tabella */
)
```

#### 4.1.4 Vincoli interrelazionali

Un vincolo di integrità referenziale (= interrelazionale) crea un legame tra i valori di un attributo (o di un insieme di attributi) A della tabella corrente (detta interna/slave) e i valori di un attributo (o di un insieme di attributi) B di un'altra tabella (detta esterna/master):

- Impone che, in ogni tupla della tabella interna, il valore di A, se diverso dal valore nullo, sia presente tra i valori di B nella tabella esterna.
- L'attributo B della tabella esterna deve essere soggetto a un vincolo UNIQUE (o PRIMARY KEY).

##### Nota:

L'attributo (o insieme di attributi) B può anche non essere la chiave primaria, ma deve essere identificante per le tuple della tabella esterna

**REFERENCES** e **FOREIGN KEY** permettono di definire vincoli di integrità referenziale

Per singoli attributi, basta utilizzare il costrutto **REFERENCES**

```
CREATE TABLE Impiegato (
 Matricola CHARACTER(6) primary key,
 Nome CHARACTER VARYING(20) not null,
 Cognome CHARACTER VARYING(20) not null,
 Dipart CHARACTER VARYING(15)
 REFERENCES Dipartimento(NomeDip),
 /* Attributo chiave (singolo) di tabella esterna */
 Ufficio NUMERIC(3),
 Stipendio NUMERIC(9) default 0,
 UNIQUE (Cognome, Nome)
)
```

Per attributi multipli, si usa il costrutto **FOREIGN KEY** alla fine della definizione degli attributi, seguito da **REFERENCES**

```
CREATE TABLE Impiegato (
 Matricola CHARACTER(6) primary key,
 Nome CHARACTER VARYING(20) not null,
 Cognome CHARACTER VARYING(20) not null,
```

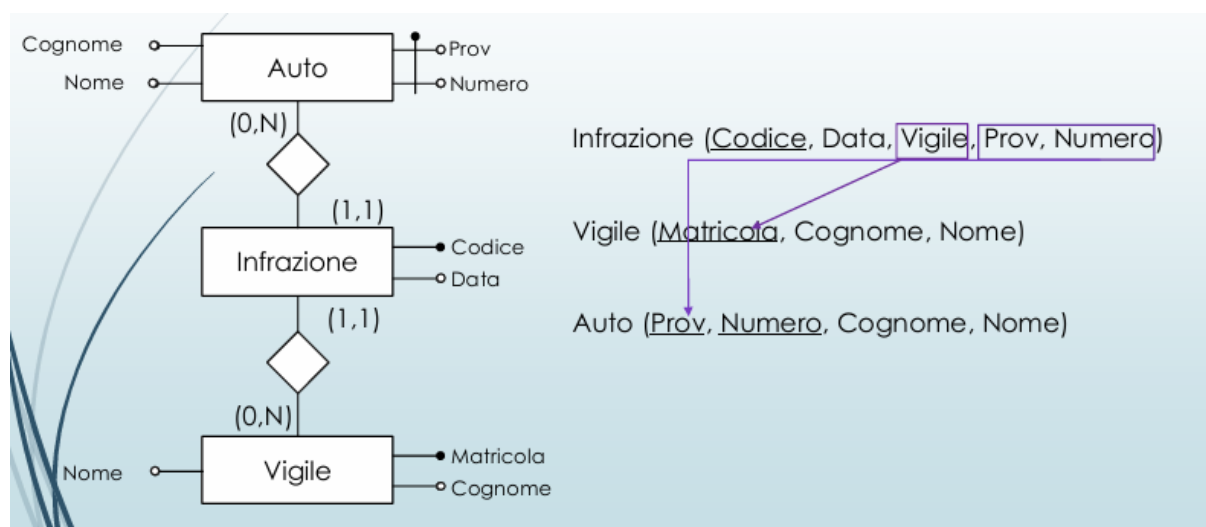


```

Dipart CHARACTER VARYING(15),
REFERENCES Dipartimento(NomeDip),
Ufficio NUMERIC(3),
Stipendio NUMERIC(9) default 0,
FOREIGN KEY(Nome, Cognome)
 REFERENCES Anagrafica (Nome, Cognome)
/* Tabella esterna con attributi chiave*/
)

```

Vediamo un esempio più concreto:



```

CREATE TABLE Infrazione (
 Codice CHAR(6) PRIMARY KEY,
 Data DATE NOT NULL,
 Vigile INTEGER NOT NULL
 REFERENCES Vigile (Matricola),
 Provincia CHAR(2),
 Numero CHAR(6),
 FOREIGN KEY (Provincia, Numero)
 REFERENCES Auto (Provincia, Numero)
)

```

## 4.2 Modifica degli schemi

- **ALTER**: effettua modifiche (a domini, tabelle...), ad esempio aggiungere una colonna a una relazione
- **DROP DOMAIN**: permette di rimuovere i domini
- **DROP TABLE**: cancella una tabella

### 4.2.1 ALTER: modifiche alla struttura di una tabella

Il costrutto **Alter** permette di:

Aggiungere un nuovo attributo (ADD COLUMN):

```
ALTER TABLE tabella ADD COLUMN attributo tipo;
ALTER TABLE impiegato ADD COLUMN stipendio NUMERIC(8,2);
```

Rimuovere un attributo (DROP COLUMN):

```
ALTER TABLE tabella DROP COLUMN attributo;
ALTER TABLE impiegato DROP COLUMN stipendio;
```

Modificare valore di default di un attributo (ALTER COLUMN):

```
ALTER TABLE tabella ALTER COLUMN attributo { SET DEFAULT valore |
DROP DEFAULT };
ALTER TABLE impiegato ALTER COLUMN stipendio SET DEFAULT 1000.00;
```

### 4.3 Cancellazione dati in una tabella

Le tuple di una tabella vengono cancellate con il comando **DELETE**. condizione è una espressione booleana che seleziona quali tuple cancellare.

```
DELETE FROM tabella [WHERE condizione];
```

**Se WHERE non è presente, tutte le tuple saranno cancellate.**

```
DELETE FROM impiegato WHERE matricola = 'A001 ';
```

Una tabella viene cancellata con il comando **DROP TABLE**:

```
DROP TABLE tabella;
```

### 4.4 Inserimento e aggiornamento di tuple

#### Inserimento di valori all'interno di una tupla

```
INSERT INTO Tabella [(Attributi)]
VALUES(Valori)
```

oppure

```
INSERT INTO Tabella [(Attributi)]
(SELECT ...)
```

#### Esempio inserimento

```
INSERT INTO Museo (nome, città, indirizzo, numeroTelefono, giornoChiusura,
VALUES('Arena', 'Verona', 'piazza Bra', '045 8003204', 'MARTEDI', 20)
```

#### Aggiornamento di valori all'interno di una tupla

```

UPDATE NomeTabella
 SET Attributo = <
 Espressione |
 SELECT ... |
 NULL |
 DEFAULT >
 [WHERE Condizione]

```

### Esempio

```
UPDATE Museo SET sitoInternet = 'www.ciao.it' WHERE citta ='Verona'
```

## 4.5 Selezione

### Struttura base

```

SELECT [DISTINCT]
[* | expression [[AS] output_name] [,
...]]
FROM from_item [, ...]
[WHERE condition]
[GROUP BY grouping_element [, ...]]
[HAVING condition [, ...]]
[{ UNION | INTERSECT | EXCEPT } [DISTINCT
] other_select]
[ORDER BY expression [ASC | DESC | USING
operator]]

```

#### 4.5.1 Clausola SELECT

```

SELECT [DISTINCT]
[{ * | expression [[AS] output_name] } [, ...]]

```

**\*** è un'abbreviazione per indicare tutti gli attributi delle tabelle.

**expression** è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).

**output\_name** è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione **expression** nella relazione risultato.

**DISTINCT**: se presente richiede l'eliminazione delle tuple duplicate.

L'esecuzione di una **SELECT** produce una relazione che:

- ha come schema tutti gli attributi elencati nella clausola **SELECT**;
- ha come contenuto tutte le tuple ottenute proiettando sugli attributi indicati nella **SELECT** provenienti dalle tabelle del **FROM** (prodotto cartesiano), limitate a quelle che soddisfano le eventuali condizioni specificate nelle clausole **WHERE**, **GROUP BY** e **HAVING**

### 4.5.1.1 Target List

```
SELECT *
FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

```
SELECT Attributo1 , Attributo2
FROM Tabella
```

Selezione di tutti gli attributi delle tabelle indicate nella clausola FROM

#### Esempio

```
SELECT Nome, Reddito
FROM Persone
WHERE Eta < 30
```

| Nome   | Reddito |
|--------|---------|
| Andrea | 21      |
| Aldo   | 15      |
| Filipo | 30      |

**Figura 4.1:** selezionati solo le tuple dove è rispettata la condizione

### 4.5.1.2 Concatenazione stringhe

L'operatore di concatenazione `||` è un operatore scalare (o operatore di riga).

Lavora su una singola riga alla volta. Se la tua tabella ha 100 righe, il costrutto titolo `|| ' - '` `||` città calcolerà e restituirà 100 risultati testuali separati, uno per ogni riga.

#### nota

Non è un'esclusiva della SELECT

può infatti essere utilizzato anche

#### Nella clausola WHERE (per filtrare i dati)

— Trova la riga in cui l'unione di nome e cognome da esattamente 'Mario  
 WHERE nome `|| ' ' ||` cognome = 'Mario Rossi'

#### Nella clausola ORDER BY (per ordinare i risultati)

— Ordina i risultati basandosi sull'unione testuale dei due campi  
 ORDER BY titolo `||` città

### 4.5.2 Clausola FROM

```
FROM from_item [, ...]
```

1. table\_name `[[AS] alias [(column_alias [, ...])]]`

- Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nel SELECT questi attributi sono identificati antepoendo il nome della tabella e un '.': nomeTabella.nomeAttributo
2. (other\_select) [AS] nomeRisultato [(column\_alias [...])]
    - è una SELECT innestata. Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
  3. from\_item [NATURAL] join\_type from\_item [ON condition]
    - è la clausola JOIN.
    - Metodo più sofisticato di 1) che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. join\_type specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti

#### 4.5.2.1 Uso di alias

```
SELECT P.Reddito , R.Telefono
FROM Reparti as R, Persona ad P
WHERE ... ;
```

Utile quando è necessario specificare la provenienza dell'attributo, perchè nella base di dati sono presenti più attributi con lo stesso nome

#### 4.5.2.2 Target List

```
SELECT P.Nome as NPersona , P.Reddito as RPersona
FROM Persone as P
WHERE P.Eta < 30
```

#### 4.5.3 Clausola WHERE

WHERE condition

**condition** è un'espressione booleana combinando condizioni semplici  $C_i$  con gli operatori AND, OR, NOT.

1.  $C_i := \text{expr} [ \theta \{ \text{expr} \mid \text{const} \} ]$  dove
  - $\text{expr} :=$  espressione che contiene riferimenti agli attributi delle tabelle che compaiono nella clausola FROM.
  - $\theta \in \{=, <, >, <=, >= \}$
  - $\text{const} :=$  valore dei domini di base.
2. Una riga soddisfa **condition** se l'espressione è vera quando valutata con i valori della riga.
3. Una riga che non soddisfa condition è scartata dal risultato finale

```
SELECT Nome, Reddito FROM Persone
WHERE Età < 30 OR Reddito > 50;
```

```
SELECT Nome, Reddito
FROM Persone
WHERE Età < 30;
```

### 4.5.3.1 Operatore LIKE

WHERE attributo [NOT] LIKE 'pattern' ;

Nella clausola WHERE può apparire l'operatore LIKE per il confronto di stringhe. LIKE è un operatore di pattern matching. I pattern si costruiscono con i caratteri speciali `_` e `%`:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.

```
SELECT Nome, Reddito
FROM Persona
WHERE Nome LIKE 'A\%a'
```

| Nome   | Reddito |
|--------|---------|
| Andrea | 21      |
| Anna   | 35      |

### 4.5.3.2 Operatore SIMILAR TO

L'operatore **SIMILAR TO** è un **LIKE** più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL). Esempi di componenti di espressioni regolari:

- `_` = 1 carattere qualsiasi.
- `%` = 0 o più caratteri qualsiasi.
- `*` = ripetizione del precedente match 0 o più volte.
- `+` = ripetizione del precedente match UNA o più volte.
- `{n,m}` = ripetizione del precedente match almeno n e non più di m volte.
- `[ ... ]` = ... è un elenco di caratteri ammissibili

```
SELECT *
FROM Persone
Per trovare nomi che
iniziano con A, F o O
WHERE Nome SIMILAR TO '[AFO]'
```

Per trovare nomi che iniziano per A,F o O

### 4.5.3.3 Operatore BETWEEN

WHERE attributo [NOT] BETWEEN valore\_iniziale AND valore\_finale ;

Nella clausola **WHERE** può apparire l'operatore **BETWEEN** per testare l'appartenenza di un valore ad un intervallo (non limitato ai valori numerici, usato anche con date e stringhe)

```
SELECT *
FROM Persone
WHERE Eta BETWEEN 20 AND 40
```

| Nome   | Età | Reddito |
|--------|-----|---------|
| Andrea | 27  | 21      |
| Aldo   | 25  | 15      |
| Filipo | 26  | 30      |
| Olga   | 30  | 41      |

#### 4.5.3.4 Operatore IN

```
WHERE attributo [NOT] IN (valore [, ...]);
```

Nella clausola WHERE può apparire l'operatore **[NOT] IN** per testare l'appartenenza di un valore ad un insieme.

```
SELECT *
FROM Persone
WHERE Nome NOT IN ('Andrea ', 'Franco ')
```

#### 4.5.3.5 Operatore IS NULL

```
WHERE attributo IS [NOT] NULL;
```

Nella clausola WHERE può apparire l'operatore **IS [NOT] NULL** per testare se un valore è **NOT KNOWN (NULL)** o no. Il valore NULL non rappresenta lo zero o una stringa vuota, ma l'assenza di dati o un valore sconosciuto. A partire dalle versioni SQL-2 è stata adottata una logica a tre valori (true, false, unknown) e i valori nulli non sono considerati né veri, né falsi.

NON SI PUÒ usare '=' o '<>' con il valore NULL!

```
SELECT *
FROM Persone
WHERE Reddito IS NOT NULL
```

#### 4.5.3.6 Operatore ORDER BY

```
ORDER BY attributo [{ ASC | DESC }] [, ...];
```

La clausola ORDER BY ordina le tuple del risultato in ordine rispetto agli attributi specificati, ASCendente è lo standard.

```
SELECT Nome
FROM Persone
ORDER BY Nome DESC
```

#### 4.5.3.7 Operatori di aggregazione

Sono operatori che permettono di determinare un valore considerando i valori ottenuti da una SELECT. Due tipi principali:

- COUNT.
- MAX, MIN, AVG, SUM.

```
SELECT MAX(Reddito)
FROM Persone;
```

**Nota**

Quando si usano gli operatori aggregati, dopo la SELECT non possono comparire espressioni che usano i valori presenti nelle singole tuple perché il risultato è sempre e solo una tupla

**1. Count**

```
COUNT({ * | expr | ALLexpr | DISTINCTexpr })
```

Restituisce il numero di tuple significative nel risultato dell'interrogazione. Tre casi comuni:

- **COUNT(\*)** ritorna il numero di tuple nel risultato dell'interrogazione.
- **COUNT(expr)** ritorna il numero di tuple in ciascuna delle quali il valore expr è non nullo
- **COUNT(ALL expr)** include duplicati e valori nulli.
- **COUNT(DISTINCT expr)** come con **COUNT(expr)** ma con l'ulteriore condizione che i valori di expr siano distinti

**2. SUM/MAX/MIN/AVG**

```
op ({ expr | DISTINCTexpr })
```

Determinano un valore numerico (SUM/AVG) o alfanumerico (MAX/MIN) considerando le tuple significative nel risultato dell'interrogazione.

- op = SUM | AVG | MAX | MIN;
- expr è un'espressione che usa uno o più attributi e funzioni di attributi.
- DISTINCT considero solo i valori distinti.

```
SELECT AVG(Eta) AS Media_Eta
FROM Persone ;
```

## 4.6 Esempi di interrogazioni

### 4.6.1 Interrogazione con raggruppamento: Group by

Un raggruppamento è un insieme di tuple che hanno medesimi valori su uno o più attributi caratteristici del raggruppamento.

La clausola GROUP BY attr [, ...] permette di determinare tutti i raggruppamenti delle tuple della relazione risultato (tuple selezionate con la clausola WHERE) in funzione degli attributi dati.

In una interrogazione che usa GROUP BY, la clausola SELECT contiene solamente gli attributi (da 0 a tutti) utilizzati per il raggruppamento ed eventuali funzioni di aggregazione sugli altri attributi.

#### Visualizzare tutte le città raggruppate della tabella Studente

```
SELECT città
FROM studente
GROUP BY città
```



| matricola | cognome | nome   | indirizzo  | città  | media   |
|-----------|---------|--------|------------|--------|---------|
| IN0001    | Rossi   | Marco  | Via nobile | Verona | 27.50   |
| IN0002    | Paoli   | Paolo  | Via dritta | Padova | 28.6666 |
| IN0003    | Bianchi | Dante  | Via storta | VERONA | 18.345  |
| IN0004    | Rossi   | Matteo | Via nobile | Verona | 24.567  |
| IN0005    | Verdi   | Luca   | Via nuova  | Va     | 28.6666 |
| IN0006    | Nocasa  | Caio   |            |        |         |

| città  |
|--------|
| Padova |
| Verona |
| VERONA |
| Va     |

## Visualizzare tutte le città e i cognomi raggruppati

```
SELECT cognome, LOWER(città)
FROM Studente
GROUP BY cognome, LOWER(città);
```

| cognome | città  |
|---------|--------|
| Verdi   | Va     |
| Rossi   | verona |
| Paoli   | padova |
| Nocasa  |        |
| Bianchi | verona |

### Nota

Nel GROUP BY non si possono usare espressioni con operatori di aggregazione.

### Nota

Nella SELECT con GROUP BY non si possono specificare attributi che non sono raggruppati.

- La clausola WHERE permette di selezionare le righe che devono far parte del risultato.
- La clausola HAVING permette di selezionare i raggruppamenti che devono far parte del risultato.
- La sintassi è HAVING bool\_expr, dove bool\_expr è un'espressione booleana che può usare gli attributi usati nel GROUP BY e/o gli altri attributi mediante operatori di aggregazione.

## Visualizzare tutte le città raggruppate che iniziano con 'V' della tabella Studente.

```
SELECT città
FROM Studente
GROUP BY città
HAVING città LIKE 'V%';
```

| Città  |
|--------|
| Verona |
| VERONA |
| Va     |

## Visualizzare tutte le città e i cognomi raggruppati con almeno due studenti con lo stesso cognome..

```
SELECT cognome, LOWER(città)
FROM Studente
GROUP BY cognome, LOWER(città)
HAVING COUNT (cognome) >1;
```

| cognome | città  |
|---------|--------|
| Rossi   | verona |

**Si possono specificare espressioni con operatori di aggregazione su attributi non raggruppati anche nella clausola HAVING.**

```
SELECT LOWER(citta) AS "Citta", AVG (media)::
FROM Studente
GROUP BY LOWER(citta)
HAVING AVG(media) >23.47 ;
```

| Città  | mediaArr |
|--------|----------|
| padova | 28.67    |
| va     | 28.67    |
| Verona | 23.47    |

AS "mediaArr."

## 4.6.2 Interrogazione con JOIN

```
TABLE1 join_typeTABLE2 ON join_condition[, ...].
```

**INNER JOIN**, **LEFT [OUTER] JOIN**, **RIGHT [OUTER] JOIN** e **FULL [OUTER] JOIN**. Dove **join\_condition** è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola **WHERE**. Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**.

### Interrogazioni con INNER JOIN Rappresenta

```
table1 [INNER] JOIN table2 ON join_condition[, ...]
```

Rappresenta il tradizionale  $\theta$  join dell'algebra relazionale. Combina ciascuna riga  $r_1$  di  $table1$  con ciascuna riga di  $table2$  che soddisfa la condizione della clausola ON. `table1 [ INNER] JOIN table2 ON join_condition[, ...]`

## 4.6.3 Interrogazioni con LEFT OUTER JOIN

```
table1 LEFT[OUTER] JOIN table2 ON join_condition[, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga  $r_1$  di  $table1$  che non soddisfa la condizione con qualsiasi riga di  $table2$ , si aggiunge una riga al risultato con i valori di  $r_1$  e assegnando NULL agli altri attributi..

Il LEFT [OUTER] JOIN non è simmetrico! Con le medesime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

## 4.6.4 Interrogazioni con RIGHT OUTER JOIN

```
table1 RIGHT[OUTER] JOIN table2 ON join_condition[, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga  $r_2$  di  $table2$  che non soddisfa la condizione con qualsiasi riga di  $table1$ , si aggiunge una riga al risultato con i valori di  $r_2$  e assegnando NULL agli altri attributi..

## 4.6.5 Interrogazioni con FULL OUTER JOIN

```
table1 FULL[OUTER] JOIN table2 ON join_condition[, ...].}
```

- È equivalente a: INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.
- **Non è equivalente a CROSS JOIN!**